FRAMEWORK FOR TRAINING AND DEPLOYMENT OF LOCOMOTION FOR HUMANOID ROBOTS BASED ON DIGITAL TWINS AND REINFORCEMENT LEARNING

by

Juan Andrés Méndez G.

Submitted to the

Department of System & Computing Engineering

in Partial Fulfillment of the Requirements for the Degree of

Masters in Systems & Computing Engineering

at the

University of the Andes

Jan 2026

© 2026 University of the Andes. All rights reserved.

Thesis Supervisors

Carlos Andrés Lozano Garzón

Ph.D. in Technology

FRAMEWORK FOR TRAINING AND DEPLOYMENT OF LOCOMOTION FOR HUMANOID ROBOTS BASED ON DIGITAL TWINS AND REINFORCEMENT LEARNING

by

Juan Andrés Méndez G.

Submitted to the
Department of Systems & Computing Engineering
on January 24, 2026 in Partial fulfillment of the
Requirements for the Degree of

Masters in Systems & Computing Engineering

Abstract:

Humanoid robot locomotion continues to pose fundamental challenges in reinforcement learning deployment, particularly regarding the simulation-to-reality transfer gap, hardware safety during experimental training, and control complexity in high-degree-of-freedom articulated systems. Despite recent advances in deep reinforcement learning and physics simulation, the transition from simulated policies to physical robots remains a critical

bottleneck in robotics research and education, where limited hardware access and damage risks significantly constrain iterative development. This thesis investigates the hypothesis that end-to-end reinforcement learning can achieve effective goal-directed navigation for high-DOF humanoid robots without hierarchical decomposition into trajectory planning and gait generation, and that simulation-to-simulation transfer using authentic communication protocols serves as a viable intermediate validation step. The research develops and validates a comprehensive framework for the Unitree G1 humanoid robot using NVIDIA Isaac Lab, implementing a custom reinforcement learning task for goal-directed point-to-point navigation using Proximal Policy Optimization (PPO) operating on proprioceptive observations across 29 actuated degrees of freedom. The framework achieves 87% goal success rate with stable bipedal locomotion, supporting the central hypothesis. A simulation-to-simulation (sim2sim) verification framework employing authentic ROS 2 Humble protocols enables policy evaluation under communication realistic conditions prior to physical deployment. Exploratory work on social navigation infrastructure—integrating LiDAR-based obstacle perception with proximity-aware reward shaping—revealed fundamental reward design challenges: the policy achieved only 1% goal success rate, demonstrating that multi-objective reward formulations for simultaneous goal-seeking and obstacle avoidance require substantial redesign, with documented analysis identifying specific gradient imbalances that inform future approaches. While systematic latency characterization and effective social navigation remain as future work, the established infrastructure provides a validated foundation for safer sim-to-real transfer in educational and research contexts where physical robot access is limited.

Keywords:

Digital Twins, Humanoid Robots, Deep Reinforcement Learning, Sim-to-Sim Validation, ROS 2, NVIDIA Isaac Lab, Unitree G1, Locomotion Control, PPO Algorithm, Social Navigation, LiDAR Perception

Acknowledgements

The completion of this thesis would not have been possible without the guidance, support, and encouragement of remarkable individuals who have shaped both my academic journey and personal growth.

I extend my deepest gratitude to my thesis supervisor and dear friend, **Dr. Carlos Andrés Lozano Garzón**. His exceptional depth of knowledge in IoT, Cybersecurity, and Metaheuristics Optimization provided the intellectual foundation for this work. Yet beyond his technical expertise, it was his profoundly humane treatment, unwavering comprehension, and patient guidance that made this journey possible. His mentorship extended far beyond what any thesis supervisor is expected to provide, and for that, I am forever grateful.

I am deeply thankful to **Dr. Germán Adolfo Montoya Orozco**, whose teaching in my first optimization course ignited a passion that ultimately guided me toward this Master's degree and my enduring interest in robotics. His ability to inspire curiosity in complex subjects left a lasting impression on my academic path.

My sincere appreciation goes to **Dr. Edgar Alejandro Marañón León**. His companionship throughout this process and his contagious enthusiasm for the applications of artificial intelligence in mechanical engineering enriched my perspective and motivated me to explore interdisciplinary approaches.

I am profoundly grateful to **Dr. Alejandra Tabares Pozos**, Assistant Professor in the Department of Industrial Engineering. Her trust and belief in my capabilities opened doors that fundamentally shaped this research. The opportunity to work closely with the Ambiente Integrado de Aprendizaje (AIA) laboratory, which she is part of, provided the essential infrastructure and collaborative environment that made this thesis possible.

Special thanks to **Álvaro Urel Achury**, laboratory assistant at the Colibri robotics facility, for his patience and willingness to share his deep knowledge of the Unitree G1 humanoid robot. His practical expertise and generous support were invaluable in bridging the gap between theoretical concepts and hands-on implementation.

To my **mother**: words cannot adequately express the depth of my gratitude for the ex-

traordinary care and love you provided throughout the countless long nights and demanding days—not only during this Master's program but throughout my entire undergraduate journey as well. Your unwavering support, quiet sacrifices, and endless encouragement have been an incalculable source of strength. This achievement is as much yours as it is mine.

To my **father**: you are my inspiration for pursuing studies in STEM. You taught me, through your example and words, that anything is possible with sufficient effort and dedication. Beyond being the first engineer I ever knew, you have always been a great human being whose integrity and work ethic continue to guide my path. Thank you for showing me what excellence truly means.

To all of you—mentors, colleagues, friends, and family—this work stands as a testament to your collective influence on my life. Thank you.

Contents

Abstract **i**

Keywords ii

Acknowledgements **iii**

Contents **v**

List of tables **ix**

List of figures **x**

Acronyms **xii**

1 Introduction **1**

2 Objectives **4**

2.1 General Objective 4

2.2 Specific Objectives 4

3 Literature Review **6**

3.1 Humanoid Robot Platforms and Market Evolution 6

3.2 Digital Twins for Robotics 7

3.3 Reinforcement Learning for Locomotion Control 9

3.3.1 Foundations of Reinforcement Learning 9

3.3.2 Contemporary Approaches to Humanoid Locomotion 10

3.4 Simulation-to-Reality Transfer Methodologies 14

3.4.1 Sources of the Reality Gap 15

3.4.2 Existing Transfer Strategies 15

3.4.3 The Validation Gap 16

3.4.4 Communication Constraints in Real Robotic Systems 17

3.4.5 Contribution of This Work 17

3.5 ROS 2 and Robotics Middleware 18
3.6 Research Gaps and Thesis Positioning 20
3.6.1 End-to-End Point-to-Point Navigation via Low-Level Joint Control 21
3.6.2 Communication-Realistic Validation Through Authentic ROS 2 Middleware 24
3.6.3 Educational Accessibility Through Containerized Deployment . . 25
3.6.4 Positioning and Future Validation 26

4 Problem Formulation **28**
4.1 Markov Decision Process Formulation 28
4.1.1 State Space $\mathcal{S}$ 28
4.1.2 Action Space $\mathcal{A}$ 29
4.1.3 Transition Dynamics P 30
4.1.4 Reward Function R 31
4.1.5 Discount Factor and Episode Horizon 32
4.2 Optimization Objective 32
4.3 Problem Characteristics and Algorithm Selection 33

5 Methodology **34**
5.1 Digital Twin Environment Creation 35
5.1.1 Unity to USD Format Conversion 35
5.1.2 Material Reconstruction and Retexturing 36
5.1.3 Physics Configuration 37
5.2 Robot Integration and Isaac Lab Extension 38
5.2.1 Unitree G1 Robot Configuration 39
5.2.2 Isaac Lab Extension Architecture 40
5.3 Custom RL Task Design for Goal-Directed Navigation 42
5.3.1 Goal Command Generation 42
5.3.2 Goal-Directed Velocity Commands 43
5.3.3 Custom Reward Functions 44
5.4 Policy Training 45

5.4.1 Training Configuration 45
5.4.2 Domain Randomization 48
5.4.3 Training Procedure 49
5.5 Simulation-to-Simulation ROS 2 Validation 50
5.5.1 Validation Architecture 50
5.5.2 ROS 2 Bridge Implementation 51
5.5.3 Communication Realism 52
5.5.4 Docker Containerization 52
5.6 Social Navigation Infrastructure (Exploratory) 53
5.6.1 LiDAR Sensor Integration 53
5.6.2 Dynamic Obstacle Infrastructure 55
5.6.3 Observation Space Extension 56
5.6.4 Avoidance Reward Framework 57
5.6.5 PPO Configuration 57

6 Results and Discussion **60**
6.1 Base Locomotion Training 60
6.1.1 Training Performance 60
6.1.2 Final Training Metrics 64
6.1.3 Evaluation Performance 64
6.1.4 Training-Evaluation Gap Analysis 65
6.1.5 Reward Component Analysis 66
6.1.6 Qualitative Observations 68
6.2 Social Navigation Training (Exploratory Results) 69
6.2.1 Training Performance 69
6.2.2 Obstacle Avoidance Learning 74
6.2.3 Final Training Metrics 77
6.2.4 Comparison with Base Locomotion Training 77
6.3 Social Navigation Evaluation (Negative Result) 77
6.3.1 Behavioral Analysis 78

6.3.2 Qualitative Observations . . . 79
6.3.3 Environment Comparison . . . 81
6.4 Sim-to-Sim ROS2 Infrastructure Verification . . . 82
6.4.1 Scope of Verification . . . 82
6.4.2 Functional Verification . . . 82
6.4.3 Topic Verification . . . 83
6.4.4 Limitations and Future Work . . . 83
6.5 Comparison with Prior Work . . . 84
6.5.1 Baseline Definitions . . . 84
6.5.2 Quantitative Comparison . . . 85
6.5.3 Analysis . . . 85
6.5.4 Limitations of Comparison . . . 86

7 Discussion **88**
7.1 Interpretation of Results . . . 88
7.1.1 Base Locomotion Success Factors . . . 88
7.1.2 Social Navigation: Multi-Objective Reward Design Challenges . 89
7.2 Limitations . . . 91
7.2.1 Statistical Limitations . . . 91
7.2.2 Methodological Limitations . . . 91
7.2.3 Scope Limitations . . . 92
7.3 Implications for Sim-to-Real Transfer . . . 93

8 Conclusions **95**
8.1 Contributions . . . 96
8.2 Limitations and Future Work . . . 96

References **97**

List of Tables

1 Comparison of Contemporary Humanoid Locomotion Approaches 14
2 State space components for base locomotion task 29
3 Domain randomization parameters 30
4 Reward function specification for base locomotion 31
5 Material Property Mapping: Unity Standard Shader to Omniverse MDL . 37
6 Unitree G1 Joint Configuration (29 DOF) 40
7 Extension Component Descriptions 41
8 Registered Gymnasium Tasks 42
9 PPO Training Hyperparameters 46
10 ROS 2 Topics Published by Sim2Sim Bridge 51
11 LiDAR sensor specifications for social navigation 54
12 Dynamic obstacle specifications 55
13 Social navigation observation space 56
14 Social navigation reward components 57
15 PPO configuration comparison: base locomotion vs. social navigation . . 58
16 Training metrics at final iteration (1,500) 65
17 Policy evaluation metrics (100 episodes) 65
18 Reward component contributions at final iteration 67
19 Social navigation training metrics at final iteration (3,000) 77
20 Training convergence comparison: base locomotion vs. social navigation 78
21 Social navigation policy evaluation metrics (100 episodes, seed=42) . . . 78
22 Comparison of base locomotion and social navigation task results 81
23 ROS2 topic verification results 83
24 Performance comparison with baselines 85

List of Figures

1 ROS 2 publish-subscribe architecture. Nodes are independent processes that communicate through typed topics. Publishers send messages without knowledge of subscribers, and subscribers receive messages without knowledge of publishers. The DDS middleware layer handles discovery, serialization, and transport, enabling distributed communication with configurable quality-of-service policies. 19

2 Hirearchical Navigation vs End-to-End approach 23

3 Methodological pipeline overview 34

4 Unity to USD conversion pipeline 36

5 Laboratorio Colivri digital twin in Isaac Sim 38

6 Unitree G1 hardware component overview 39

7 Isaac Lab extension architecture 41

8 Forward-biased goal sampling distribution 43

9 PPO actor-critic network architecture 47

10 PPO training procedure 50

11 Sim2sim validation architecture 51

12 Sim2sim validation in RViz2 53

13 Social navigation architecture 54

14 Proximity penalty zones 58

15 Base locomotion: episode reward 61

16 Base locomotion: episode length 62

17 Base locomotion: PPO training losses 63

18 Base locomotion: distance to goal 64

19 Base locomotion: policy evaluation 68

20 Social navigation: episode reward 70

21 Social navigation: episode length 71

22 Social navigation: termination analysis 72

23 Social navigation: PPO training losses 73

24 Social navigation: obstacle penalties 74
25 Social navigation: distance to goal 75
26 Social navigation: goal reached rate 76
27 Social navigation: policy evaluation 80

Acronyms

RL Reinforcement Learning

DRL Deep Reinforcement Learning

ROS Robot Operating System

DOF Degrees of Freedom

GPU Graphics Processing Unit

PPO Proximal Policy Optimization

URDF Unified Robot Description Format

USD Universal Scene Description

1 Introduction

The field of humanoid robotics is undergoing significant transformation in 2023-2026, characterized by substantial cost reductions, advanced integration of artificial intelligence integration, and expanding educational applications. The humanoid robotics market, projected to reach $38 billion by 2035, is experiencing rapid changes driven by platforms that reduce costs by up to 90% compared to previous generations [1]. This shift is reshaping how educational and research institutions approach robotics, creating opportunities for universities to engage with advanced technology that was previously financially inaccessible.

Humanoid robots represent one of the most complex and promising domains in contemporary engineering, designed to navigate and interact within environments originally conceived for humans. These platforms face substantial challenges in design, control, and programming derived from their elevated kinematic and dynamic complexity [2]. The development of robust behaviors has historically been constrained by prohibitive hardware costs, system vulnerability during experimental training, and the difficulty of controlling devices with dozens of degrees of freedom [3]. These limitations have created significant barriers for educational institutions seeking to incorporate advanced robotics into their curricula and research programs.

The Laboratorio Colivri (Collaboration Laboratory for Interaction, Visualization, Robotics and Autonomous Systems) at the Universidad de los Andes exemplifies these challenges on a scale. Since 2025, the laboratory has had single Unitree G1 humanoid robot to serve the educational and research needs of more than 8,000 engineering students. This situation creates three critical operational challenges that demand innovative solutions:

First, the **scalability and logistics** challenge emerges from having only one robot available for an entire academic community. Programming practices, projects, and demonstrations are severely limited, with no efficient mechanism to guarantee equitable usage time between courses and research groups. The planned exhibition activities—where the G1 must navigate the laboratory and demonstrate capabilities to visitors—compete directly with teaching schedules and development time, creating severe operational bot-

tlenecks that limit the robot's educational impact.

Second, the **safety and preservation of hardware** are in paramount, given that the G1 represents a substantial investment for the laboratory. The robot, with its approximately 35 degrees of freedom and complex contact dynamics, faces inherent risks when new locomotion routines are tested directly on the physical hardware [4]. Damage from falls, overcurrents, or control instabilities would result in elevated repair costs and extended downtime periods, potentially disrupting entire academic semesters.

Third, the **advanced functional requirements** imposed as purchase conditions demand sophisticated capabilities. The robot must perform exhibition tasks including navigating autonomously through the laboratory and demonstrating various locomotion behaviors. These requirements necessitate robust navigation policies capable of goal-directed movement and dynamic balance—capabilities that traditionally require extensive training and validation cycles.

Recent technological advances offer a promising framework to address these challenges. The emergence of high-fidelity physical simulation platforms, such as NVIDIA Omniverse Isaac Sim, enables the creation of digital twins capable of accurately reproduce contact dynamics, friction, and material deformation with high precision [5]. Deep reinforcement learning techniques have demonstrated remarkable effectiveness in synthesizing control policies for systems with multiple degrees of freedom, particularly when employing hierarchical approaches, progressive curricula, or model-based methods [6]. Furthermore, the integration of standardized robotics middleware like ROS 2 provides reliable communication protocols that can bridge simulation and physical deployment environments.

The convergence of these technologies—accessible hardware platforms, advanced simulation environments, and proven reinforcement learning methods—creates a significant opportunity to increase accessibility of advanced robotics education. Unitree G1, at approximately $16,000 for basic configurations, represents a 90% cost reduction over previous-generation humanoids, making it accessible to institutions previously priced out of humanoid robotics research. With 23-43 degrees of freedom depending on configuration, integrated NVIDIA Jetson Orin processing delivering 100 TOPS of AI capability,

and comprehensive sensor suites including depth cameras and 3D LiDAR, the G1 provides a capable platform for exploring advanced locomotion control.

This thesis proposes the development of an integrated methodological framework that combines two core technological components—digital twins and deep reinforcement learning to safely and efficiently train and validate locomotion policies for the Unitree G1. The framework addresses the fundamental barriers that have limited progress in humanoid robotics while providing a scalable solution for educational institutions facing similar challenges. Through the construction of a high-fidelity digital twin of both the robot and its operational environment, the methodology enables extensive experimentation without physical risks. By applying deep reinforcement learning with Proximal Policy Optimization, the system facilitates the synthesis of robust locomotion and navigation policies. Critically, by implementing a simulation-to-simulation validation environment that uses authentic ROS 2 communication protocols, the framework provides a credible intermediate step before costly physical deployment, substantially mitigating the simulation-to-reality transfer gap.

The central hypothesis of this research is that end-to-end reinforcement learning can achieve effective goal-directed navigation for high-DOF humanoid robots without requiring hierarchical decomposition into separate trajectory planning and gait generation modules, and that simulation-to-simulation transfer using authentic communication protocols can serve as a viable intermediate validation step before physical deployment. This hypothesis challenges the conventional approach to humanoid locomotion, which that typically decomposes the problem into distinct layers—high-level path planning, mid-level trajectory generation, and low-level gait control—by demonstrating that a single learned policy operating directly on joint-level actions can produce stable, goal-directed locomotion. Additionally, the research explores whether LiDAR-based obstacle perception can be integrated into such policies for social navigation, documents the design challenges that emerge from multi-objective reward formulations and informs future approaches to human-aware humanoid locomotion.

The proposed methodology seeks to achieve multiple interconnected objectives: significantly reducing hardware damage risk and associated costs; accelerating iteration cy-

cles through exhaustive simulation validation; validating learned policies under realistic communication constraints; and enabling broader robot access among the thousands of students who use the Laboratorio Colivrí. While the study focuses on the Unitree G1 within a specific institutional context, the approach is deliberately generalizable, intended to lay the foundations for other institutions to adopt similar solutions with different humanoid platforms and varied operational contexts. This research thus offers a scalable contribution to the fields of humanoid robotics and systems engineering, addressing the fundamental barriers that have limited the progress of these highly articulated systems in educational settings.

2 Objectives

2.1 General Objective

Develop a framework for training and validating locomotion policies for the G1 humanoid robot using digital twins and deep reinforcement learning, achieving goal-directed navigation capabilities while enabling safe policy development in simulation and validation through simulation-to-simulation transfer using ROS with a preliminary exploration of social navigation infrastructure to establish foundations for future research.

2.2 Specific Objectives

- Create a high-fidelity digital twin of the Unitree G1 humanoid robot in NVIDIA Isaac Lab, accurately incorporating its physical, kinematic, and sensorial properties to enable safe and scalable experimentation.

- Design and implement a deep reinforcement learning task using Proximal Policy Optimization (PPO) for goal-directed point-to-point navigation through low-level joint control across 29 DOF, achieving stable bipedal locomotion with high goal success rates.

- Develop a simulation-to-simulation (sim2sim) validation framework that employs

authentic ROS 2 Humble communication protocols, enabling policy evaluation under realistic message passing patterns and asynchronous communication constraints before physical deployment.

- Explore infrastructure for human-aware locomotion as preliminary groundwork for future research, implementing LiDAR sensor integration, dynamic obstacle simulation representing pedestrians, and graduated proximity penalty functions, while documenting the technical challenges encountered that inform subsequent reward design improvements.
- Validate the trained locomotion policies in the sim2sim environment, evaluating performance metrics including locomotion stability, goal-reaching accuracy, and policy robustness under ROS 2 transport layer constraints.

3 Literature Review

The development of autonomous humanoid robots capable of adaptive locomotion requires integrating multiple technological domains: robotic platforms, digital twin methodologies, reinforcement learning algorithms, simulation-to-reality transfer techniques, and communication middleware. This literature review examines recent advances in each domain, identifies critical gaps, and positions this thesis within the current research landscape.

3.1 Humanoid Robot Platforms and Market Evolution

The humanoid robotics market has experienced substantial growth, driven by technological advances and cost reductions that have increased access to sophisticated platforms. Market analysts project substantial expansion, with Goldman Sachs forecasting the global humanoid robot market could reach $38 billion by 2035 [7], while MarketsandMarkets estimates growth from $1.8 billion in 2023 to $13.8 billion by 2028 at a compound annual growth rate of 50.3% [1].

This rapid market expansion reflects a fundamental shift from expensive research platforms to commercially viable products. Contemporary humanoid platforms demonstrate increasing accessibility: Boston Dynamics' Atlas Electric showcases advanced hydraulic-free actuation; Figure AI's Figure 02 integrates vision-language models for manipulation tasks; Tesla's Optimus Gen 2 emphasizes manufacturing scalability; 1X Technologies' NEO targets consumer applications; and Apptronik's Apollo focuses on logistics and warehouse automation. Notably, the cost of capable humanoid platforms has decreased approximately 90% over the past decade, with current educational-grade platforms available at price points accessible to universities and research institutions.

The Unitree G1 humanoid robot, central to this thesis, exemplifies trend toward accessibility . With 29 degrees of freedom distributed across hip (3-DOF), knee (1-DOF), ankle (2-DOF), waist (3-DOF), shoulder (3-DOF), elbow (1-DOF), and wrist (1-DOF) joints, the G1 provides sufficient kinematic complexity for research-grade locomotion and ma-

nipulation tasks [4]. The platform integrates an NVIDIA Jetson Orin onboard computer, enabling local neural network inference, and incorporates multiple sensor modalities, including 3D LiDAR, stereo depth cameras, and a 6-axis inertial measurement unit. This combination of affordability, computational capability, and sensor integration makes the G1 particularly suitable for educational contexts where hardware budgets are constrained, but research requirements remain demanding.

The proliferation of affordable humanoid robots creates an urgent demand for safe, scalable training methodologies suitable for educational contexts where hardware access is severely limited. A single physical robot must often serve hundreds or thousands of students, making destructive training approaches—such as direct trial-and-error learning on hardware—economically and pedagogically infeasible. This constraint motivates the development of intermediate validation frameworks that reduce hardware damage risks while maintaining research-grade capabilities.

3.2 Digital Twins for Robotics

Digital twins represent virtual replicas of physical systems that enable safe experimentation, analysis, and validation without the risks and costs associated with physical prototypes. Hasan [8] define digital twins as dynamic virtual models that continuously synchronize with their physical counterparts through bidirectional data exchange, enabling prediction, optimization, and decision-making. This bidirectionality distinguishes digital twins from conventional simulations: while simulations model hypothetical scenarios, digital twins maintain ongoing correspondence with real-world systems.

The creation of accurate digital twins requires systematic physical-to-virtual twinning processes. Sturm et al. [9] identify three critical dimensions: geometric accuracy (capturing physical dimensions and spatial relationships), kinematic fidelity (representing joint configurations and constraints), and dynamic realism (modeling mass properties, friction, and contact behaviors). In robotics, this twinning process typically involves CAD-to-simulation pipelines that convert mechanical designs into physics-compatible representations, sensor modeling that replicates measurement characteristics and noise profiles, and

actuator characterization that captures torque limits, response delays, and control bandwidth.

ROS-based digital twin architectures have emerged as a dominant paradigm for robotic systems. Sueldo et al. [10] demonstrate digital twin implementations using ROS 2 for mobile robots, emphasizing the middleware's capabilities for distributed communication, quality-of-service guarantees, and real-time data transport. Their approach employs authentic communication protocols in simulation, enabling validation of control algorithms under realistic message-passing patterns, network latencies, and asynchronous communication constraints—factors often neglected in pure physics simulation.

NVIDIA Isaac Sim and Isaac Lab represent state-of-the-art platforms for creating high-fidelity robotic digital twins [5, 11]. Isaac Sim provides GPU-accelerated physics simulation using the PhysX 5 engine, photorealistic rendering via RTX ray tracing, and accurate sensor models including LiDAR, RGB-D cameras, and IMUs. Isaac Lab extends this foundation with a Python-based framework specifically designed for robot learning, offering reinforcement learning environment abstractions compatible with Gymnasium standards, efficient vectorized simulation enabling thousands of parallel environments on a single GPU, and domain randomization utilities for sim-to-real transfer. The platform supports Universal Scene Description (USD) format scenes, enabling complex environments with realistic visual and physical properties.

Digital twins serve multiple purposes in robotics research: algorithm development and testing in risk-free environments, parameter tuning without hardware wear, failure mode analysis under extreme conditions, and scalable training through massively parallel simulation. However, a critical question remains to be addressed in existing digital twin work: how can policies trained in simulation be credibly validated before deployment to physical hardware? Most frameworks assume sufficient domain randomization or direct hardware testing, leaving a credibility gap for educational contexts where hardware damage risks must be minimized. Currently, frameworks integrate digital twins with robust intermediate validation steps before physical deployment, particularly for educational contexts where thousands of students must share limited hardware resources.

3.3 Reinforcement Learning for Locomotion Control

Reinforcement learning has emerged as the dominant paradigm for training adaptive locomotion controllers, enabling robots to learn complex behaviors through interaction with the environment. This section examines foundational RL principles before surveying contemporary approaches to humanoid locomotion.

3.3.1 Foundations of Reinforcement Learning

Reinforcement learning formalizes the control problem as a Markov Decision Process (MDP) comprising states, actions, transition dynamics, rewards, and a discount factor [12]. The objective is to learn a policy that maximizes expected cumulative discounted reward. For locomotion control, states typically include joint positions, velocities, angular velocities, and base orientation; actions correspond to joint torque commands or position targets; and rewards encode task objectives such as forward velocity, energy efficiency, and stability.

Policy optimization methods directly learn parametrized policies, with Proximal Policy Optimization (PPO) demonstrating particular effectiveness for robotic control. PPO constrains policy updates to a trust region defined by a clipped objective function, preventing destructively large updates while maintaining sample efficiency. This stability proves critical for physical systems where catastrophic forgetting or unsafe policies could damage hardware. PPO has become the de facto standard for legged robot locomotion due to its balance of sample efficiency, stability, and implementation simplicity.

Simulation-to-reality transfer remains a fundamental challenge in RL-based robotics. Policies trained in simulation often fail when deployed to physical systems due to dynamics mismatch (imperfect modeling of friction, contact, and inertia), sensor noise (simulation sensors providing noiseless measurements), actuator delays (real motors exhibiting bandwidth limitations), and unmodeled effects (cable dynamics, backlash, temperature dependencies). Domain randomization addresses these challenges by training over distributions of simulation parameters—varying masses, friction coefficients, joint damping, sensor noise levels, and control frequencies—encouraging policies to develop robust be-

haviors that generalize across parameter variations.

3.3.2 Contemporary Approaches to Humanoid Locomotion

Recent research demonstrates multiple viable strategies for humanoid locomotion, each with distinct tradeoffs in terms of data requirements, computational demands, infrastructure needs, and deployment accessibility.

Motion Capture and Retargeting Approaches Cheng et al. [13] present a whole-body control framework for humanoid robots that leverages motion capture data to achieve natural, expressive movements. Their approach employs hierarchical control: a high-level policy tracks reference motions from human motion capture datasets, while a low-level controller resolves whole-body dynamics and maintains balance. This architecture enables behaviors such as expressive gestures, dynamic walking with arm swings, and socially appropriate movements suitable for human-robot interaction contexts.

Motion capture-based methods offer significant advantages for applications requiring natural-looking behaviors, such as entertainment robotics, service robots in social settings, and telepresence systems. The learned policies can reproduce complex human movements with high fidelity, providing immediate visual appeal and intuitive motion characteristics. However, these approaches demand substantial infrastructure: motion capture systems with multiple synchronized cameras, human subjects performing relevant behaviors, and motion retargeting pipelines that map human kinematic structures to robot configurations. This infrastructure requirement limits accessibility for educational institutions without dedicated motion capture laboratories.

Transformer-Based In-Context Adaptation Radosavovic et al. [2] demonstrate real-world humanoid locomotion using a causal transformer-based controller capable of in-context adaptation, published in Science Robotics. Their approach trains a large-scale transformer model on diverse simulation experiences, enabling the policy to adapt to novel terrain and disturbances without gradient updates. The controller was deployed to the Digit humanoid robot, achieving zero-shot transfer to outdoor environments and

demonstrating adaptation to slopes, uneven terrain, and external pushes.

The key innovation lies in the transformer's ability to leverage sequential context: by processing recent state-action-reward histories, the model infers environmental characteristics and adjusts behavior online. This in-context learning mimics meta-learning, where the model "learns how to learn" during deployment rather than requiring offline retraining. Radosavovic et al. [14] extend this work to challenging terrain navigation, demonstrating locomotion over obstacles, stairs, and steep inclines.

Transformer-based approaches represent the current state of the art in adaptability and robustness. The models generalize remarkably well to conditions absent from training data, suggesting strong inductive biases learned during large-scale training. However, these capabilities come with substantial computational costs: training requires massive GPU clusters (often hundreds of GPUs for weeks), the models have millions to billions of parameters demanding significant inference computation, and the training data requirements span diverse simulation scenarios necessitating extensive environment engineering. These resource demands place transformer-based methods beyond the reach of most educational institutions, limiting their applicability for student-led research projects.

Direct Reinforcement Learning Approaches Gu et al. [3] present Humanoid-Gym, an open-source reinforcement learning framework specifically designed for humanoid robot locomotion with proven zero-shot sim-to-real transfer capability. Built on NVIDIA Isaac Gym, the framework supports multiple Unitree platforms including H1, G1-29dof, G1-23dof, and B2 robots. The approach employs Proximal Policy Optimization with privileged information during training: the policy has access to ground-truth state information in simulation. However, it relies solely on proprioceptive sensing (joint encoders, IMU) during deployment.

Key methodological components include domain randomization over physical parameters (mass distributions, friction coefficients, joint damping, motor strengths), observation noise injection simulating sensor imperfections, control frequency randomization matching real hardware variability, and careful reward shaping balancing task objectives (forward velocity tracking) with regularization terms (energy efficiency, smooth actions,

minimal joint accelerations). The authors demonstrate successful deployment to physical Unitree H1 and G1 robots, validating the framework's sim-to-real transfer capabilities.

Direct RL approaches using PPO offer a compelling balance of accessibility, reproducibility, and proven transfer capability. The training can be conducted on a single workstation with a modern GPU (e.g., NVIDIA RTX 4090 or A100). The algorithms are well-documented with open-source implementations, and the methodology has been validated across multiple physical platforms. This accessibility makes direct RL particularly suitable for resource-constrained educational contexts where students require hands-on experience with state of the art methods but lack access to massive computational infrastructure.

However, existing direct RL frameworks assume zero-shot sim-to-real transfer through domain randomization alone, directly deploying simulation-trained policies to physical hardware after training completion. This approach introduces risk: if domain randomization proves insufficient, the first deployment attempt could damage expensive hardware. For educational contexts where a single robot must serve thousands of students, this risk profile is unacceptable.

Vision-Language-Action Models Emerging multimodal approaches integrate vision, language, and action learning for high-level task planning. NVIDIA's GR00T (Generalist Robot 00 Technology) framework enables robots to learn from multimodal demonstrations by combining visual observations with natural-language instructions. Mon Williams et al. [15] present ELLMER, a framework using GPT-4 for adaptive manipulation tasks, demonstrating language-conditioned policy selection and task decomposition.

Vision-language-action models show promise for high-level reasoning and task planning, they currently remain complementary to low-level locomotion control rather than replacing it. Locomotion requires real-time control at high frequencies (50-1000 Hz), demands fast inference latencies incompatible with large language model queries, and benefits from reactive behaviors that bypass deliberative reasoning. This work focuses on low-level locomotion control, acknowledging that future integration with high-level VLA planners represents a promising research direction.

Critical Comparison and Positioning The surveyed approaches reveal fundamental tradeoffs between adaptability, accessibility, and deployment risk:

- **Transformer-based methods** [2, 14] achieve remarkable adaptability and robustness through in-context learning but require substantial computational resources (hundreds of GPUs, weeks of training) beyond typical educational budgets.
- **Motion capture approaches** [13] produce natural, expressive behaviors suitable for social robotics but demand specialized motion capture infrastructure unavailable in most educational settings.
- **Direct RL methods** [3] offer accessibility, reproducibility, and proven sim-to-real capability using standard workstation hardware, making them suitable for educational contexts. However, they assume zero-shot transfer through domain randomization alone, introducing hardware damage risks if randomization proves insufficient.

Table 1 summarizes these tradeoffs quantitatively, highlighting the accessibility advantages of direct RL approaches while identifying the validation gap this thesis addresses.

Table 1: Comparison of Contemporary Humanoid Locomotion Approaches

Dimension	**Transformer-Based** [2]	**Motion Capture** [13]	**Direct RL** [3]
Adaptability	Very High (in-context learning)	Medium (retargeted motions)	Medium-High (learned policies)
Infrastructure Required	100+ GPU cluster	Motion capture lab	Single GPU workstation
Training Time	Weeks	Days-Weeks	6-12 hours
Inference Cost	High (millions of parameters)	Medium (hierarchical)	Low (compact network)
Educational Accessibility	Very Low (research labs only)	Low (specialized labs)	High (standard hardware)
Validation Strategy	Direct sim-to-real	Physical testing	Direct sim-to-real with domain randomization
Hardware Damage Risk	Medium	Low (tested motions)	Medium-High
Primary Task	Velocity tracking + terrain adaptation	Expressive whole-body motions	Velocity tracking

This thesis adopts the direct RL approach following Gu et al. [3] for its accessibility and reproducibility, but introduces an intermediate sim-to-sim validation step using authentic ROS 2 communication protocols. This methodological contribution addresses the credibility gap between pure simulation training and costly physical deployment, providing an additional verification layer particularly valuable for educational institutions with limited hardware access. While domain randomization prepares policies for physics variations, authentic communication protocols validate performance under realistic message-passing patterns, latencies, and asynchronous transport characteristics—factors critical for real-world deployment but absent from pure simulation training.

3.4 Simulation-to-Reality Transfer Methodologies

Simulation-to-reality (sim-to-real) transfer addresses the challenge of deploying policies trained in simulation to physical robots. Despite advances in simulation fidelity, system-

atic differences between virtual and real environments—termed the ”reality gap”—can cause trained policies to fail during physical deployment.

3.4.1 Sources of the Reality Gap

The reality gap arises from multiple sources. Dynamics mismatch occurs when simulation engines imperfectly model friction, contact mechanics, joint flexibility, and inertial properties. Sensor discrepancies arise when simulated sensors provide noiseless, geometrically perfect measurements while real sensors exhibit noise, drift, calibration errors, and sampling artifacts. Actuator differences manifest through unmodeled delays, bandwidth limitations, backlash, and torque ripples. Environmental factors such as air resistance, temperature effects, cable dynamics, and electromagnetic interference remain difficult to simulate accurately. Each source contributes to policy degradation during the transition from simulation to reality.

3.4.2 Existing Transfer Strategies

Current sim-to-real methodologies employ several strategies to bridge the reality gap. System identification techniques involve characterizing physical robot parameters through experiments and updating simulation models to match measured behaviors. While this approach reduces dynamic mismatch, it requires physical hardware access and extensive calibration procedures, making it unsuitable for contexts with limited hardware availability.

Domain randomization has emerged as the dominant strategy for learning robust policies [3]. By training over distributions of simulation parameters—randomly varying masses, dimensions, friction coefficients, joint damping, motor gains, sensor noise characteristics, and control frequencies—policies learn behaviors that generalize across parameter variations. The policy never encounters the ”correct” simulation parameters during training, but instead develops robust strategies that tolerate parameter uncertainty. When deployed to the physical system (which represents one sample from the parameter distribution), the policy often performs successfully despite imperfect simulation fidelity.

Progressive training strategies introduce curriculum learning in which simulation fidelity gradually increases, noise levels approach real sensor characteristics, and task difficulty incrementally. These methods aim to guide policy learning through increasingly realistic conditions, though they require careful curriculum design and extend training duration.

3.4.3 The Validation Gap

Existing sim-to-real methodologies share a common limitation: they assume direct transfer from simulation training to physical deployment, with domain randomization providing sufficient robustness. The typical workflow involves training on simulated data with domain randomization, validating performance across randomized simulation parameters, and then deploying directly to physical hardware. This approach treats physical deployment as the first and only verification step under real-world conditions.

This direct transfer paradigm introduces significant risks, particularly for expensive or fragile hardware. If domain randomization proves insufficient—due to unmodeled effects, inadequate parameter ranges, or systematic simulation biases—the first deployment attempt could result in falls, collisions, or joint damage. For research laboratories with multiple robots and dedicated technical staff, these risks can be managed through careful monitoring and emergency stops procedures. However, for educational institutions where a single $20,000 humanoid robot must serve thousands of students across multiple years, even a single damaging incident represents an unacceptable setback to the educational program.

Some research explores simulation-to-simulation (sim-to-sim) transfer as an intermediate step, validating policies trained in one simulator (e.g., Isaac Gym) by deploying to a different simulator with alternative physics engines (e.g., MuJoCo). These approaches evaluate policy robustness to physics modeling differences but typically ignore communication layer realism. The policies are evaluated through direct function calls within the target simulator, bypassing message passing protocols, network latencies, serialization overheads, and asynchronous transport characteristics present in real robotic systems.

3.4.4 Communication Constraints in Real Robotic Systems

Real robotic systems employ middleware frameworks for communication between sensing, control, and actuation components. ROS 2 Humble, the current long-term support release of the Robot Operating System, provides a distributed communication architecture based on Data Distribution Service (DDS) middleware. In this architecture, sensor data, state estimates, and control commands flow through topic-based publish-subscribe patterns rather than direct function calls.

This communication architecture introduces constraints absent from pure simulation environments. Message serialization and deserialization impose computational overheads; network transport introduces variable latencies (typically 1-10ms for local communication, potentially higher for distributed systems); asynchronous publishing creates temporal misalignment between sensor measurements and control decisions; and quality-of-service (Qos) policies affect message delivery guarantees and timing. A policy trained and validated only through direct simulation function calls has never experienced these communication characteristics, potentially leading to degraded performance or instability when deployed through real middleware.

3.4.5 Contribution of This Work

This thesis introduces a simulation-to-simulation validation framework that employs authentic ROS 2 Humble communication protocols as an intermediate verification step before physical deployment. Unlike existing sim-to-sim approaches that focus solely on physics engine differences, this work replicates realistic communication constraints by publishing sensor states through ROS 2 topics at authentic frequencies (50 Hz for joint states, 10 Hz for LiDAR point clouds), subscribing to control commands through standard Unitree message types, introducing communication latencies characteristic of real-time middleware, and maintaining temporal synchronization through ROS 2 clock mechanisms.

The framework implements a second simulation environment that receives high-level velocity commands and goal positions, publishes robot state information via Unitree-

compatible topics, and validates policy behavior under communication-realistic conditions. By evaluating policies under realistic message-passing patterns before physical deployment, this approach provides an additional credibility layer that substantially reduces hardware damage risks.

This methodology proves particularly valuable for educational contexts where hardware access is severely limited. Students can develop, train, and validate locomotion policies entirely in simulation, progressing through two verification stages (physics-based validation with domain randomization, followed by communication-realistic validation with ROS 2 middleware) before requesting precious physical robot time. This staged validation approach maximizes learning opportunities while minimizing hardware risks, enabling students to engage with humanoid robotics research using shared physical resources.

3.5 ROS 2 and Robotics Middleware

Robot Operating System 2 (ROS 2) represents the current standard for robotics middleware, providing communication infrastructure, hardware abstraction, and software modularity for robotic systems [16]. Understanding ROS 2's architecture and communication patterns proves essential for implementing communication-realistic validation frameworks.

ROS 2 employs a distributed publish-subscribe architecture based on the Data Distribution Service (DDS) standard. Unlike ROS 1's centralized master node, ROS 2 enables peer-to-peer discovery where nodes automatically detect each other without central coordination. This architecture improves fault tolerance, scalability, and real-time performance. Multiple DDS implementations are supported, including Fast-DDS, CycloneDDS, and Connext DDS, with CycloneDDS often preferred for real-time applications due to its low latency and efficient memory usage.

Figure 1 illustrates the fundamental ROS 2 communication architecture. Nodes represent independent processes that perform specific tasks (sensing, control, actuation), while topics serve as named channels through which nodes exchange typed messages. Pub-

lishers send data to topics without knowledge of which nodes will receive it; subscribers receive data from topics without knowing which nodes produced it. This decoupling enables modular system design where components can be developed, tested, and replaced independently. The DDS middleware layer handles all communication details including discovery, serialization, transport, and quality-of-service enforcement.

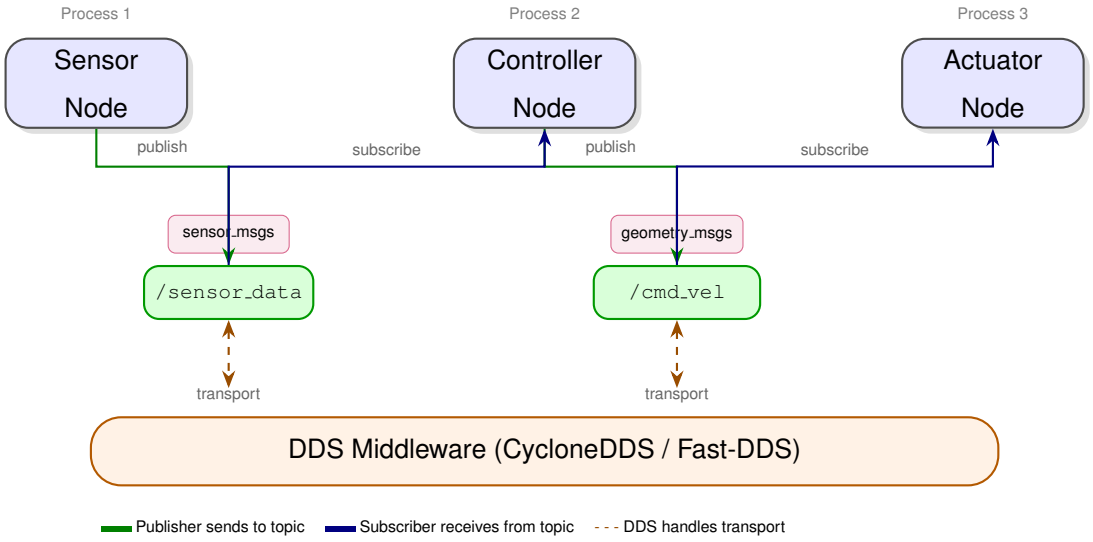


Figure 1: ROS 2 publish-subscribe architecture. Nodes are independent processes that communicate through typed topics. Publishers send messages without knowledge of subscribers, and subscribers receive messages without knowledge of publishers. The DDS middleware layer handles discovery, serialization, and transport, enabling distributed communication with configurable quality-of-service policies.

Communication occurs through typed topics: publishers send messages to named topics while subscribers receive those messages, with DDS handling transport, discovery, and quality-of-service enforcement. Message types follow standardized definitions (e.g., `sensor_msgs/JointState`, `nav_msgs/Odometry`, `geometry_msgs/Twist`) ensuring interoperability across platforms and vendors. Quality-of-service policies control reliability (reliable vs. best-effort delivery), durability (transient vs. volatile message retention), history depth (number of messages buffered), and deadline constraints (maximum acceptable time between messages).

Unitree humanoid robots employ specific ROS 2 communication patterns that any

validation framework must replicate. The `/lowstate` topic publishes joint positions, velocities, torques, and IMU data at 500-1000 Hz, providing high-frequency proprioceptive feedback. The `/lowcmd` topic subscribes to joint position or torque commands at matching frequencies, enabling low-level motor control. Sensor topics such as `/utlidar/cloud` publish 3D LiDAR point clouds at 10 Hz, while camera topics provide RGB-D images for perception tasks. This multi-rate communication structure requires careful temporal synchronization to maintain control stability.

Sueldo et al. [10] demonstrate ROS-based digital twin implementations for mobile robots, emphasizing the importance of authentic communication protocols for credible validation. Their work shows that digital twins employing actual ROS messaging exhibit more realistic behavior than those using direct function calls, as the communication layer introduces latencies, buffering effects, and serialization overheads characteristic of deployed systems.

This thesis leverages ROS 2 Humble (the long-term support release with maintenance through 2027) with CycloneDDS middleware to implement communication-realistic validation. By replicating Unitree-compatible topic structures, message types, and publication frequencies in simulation, the framework enables policies to be evaluated under authentic communication constraints before physical deployment. This approach addresses the communication validation gap identified in existing sim-to-real methodologies, providing an intermediate verification step that assesses policy performance under realistic message passing patterns, latencies, and asynchronous transport characteristics.

3.6 Research Gaps and Thesis Positioning

Consider a typical educational robotics laboratory where a single $20,000 Unitree G1 humanoid robot must serve thousands of students across multiple years. Existing sim-to-real methodologies train policies with domain randomization and deploy directly to physical hardware [3], treating hardware as the first validation under real-world conditions. If the policy fails—due to insufficient randomization, unmodeled dynamics, or communication issues—the robot falls, potentially damaging expensive actuators or structural

components. The educational program stalls while repairs are completed or replacement hardware is procured, denying hundreds of students access to hands-on robotics research.

This scenario reveals critical gaps in current humanoid locomotion research: the lack of credible intermediate validation before physical deployment, the assumption of ready hardware access for iterative testing, and the absence of communication-realistic verification despite real robots operating through middleware frameworks like ROS 2. Educational institutions cannot afford the risks inherent in direct sim-to-real transfer, yet state-of-the-art alternatives demand resources beyond typical budgets—transformer-based methods require hundreds of GPUs for weeks [2], while motion capture approaches need specialized laboratory infrastructure [13].

This thesis addresses these gaps through an integrated framework combining novel task design, communication-realistic validation, and accessible deployment strategies. Rather than listing contributions abstractly, this section demonstrates how the work extends existing methodologies to serve educational contexts where hardware access is severely constrained.

3.6.1 End-to-End Point-to-Point Navigation via Low-Level Joint Control

Traditional approaches to humanoid navigation employ hierarchical architectures: a high-level planner generates waypoints or footstep plans, a mid-level controller computes joint trajectories, and a low-level controller tracks those trajectories [13]. This decomposition simplifies the problem but introduces challenges: trajectory optimization becomes computationally expensive, footstep planning requires accurate terrain models, and coordination between hierarchy levels demands careful tuning. Each abstraction layer accumulates errors and delays, potentially degrading performance.

This work adopts a fundamentally different strategy: end-to-end point-to-point navigation through direct reinforcement learning of low-level joint control across 29 degrees of freedom. The policy receives goal positions and current robot state, then directly outputs joint position targets without intermediate trajectory generation or footstep planning. This approach eliminates hierarchical abstractions, enabling the policy to discover efficient locomotion strategies through environmental interaction rather than relying on

hand-engineered motion primitives. The choice of end-to-end learning over hierarchical decomposition is deliberate: while hierarchical architectures can provide interpretable intermediate representations and may facilitate modular development, recent work in legged robotics has demonstrated that end-to-end approaches achieve competitive or superior performance while substantially reducing engineering complexity [3]. For educational contexts where maintainability and reproducibility are paramount, architectural simplicity provides significant advantages.

The benefits prove substantial. Training becomes simpler: a single neural network replaces multiple coordinated controllers, reducing architectural complexity and hyperparameter tuning. The policy learns adaptive behaviors: rather than following pre-computed trajectories, the controller continuously adjusts joint commands based on current state, enabling reactive responses to disturbances. Computational efficiency improves: inference requires a single forward pass through a compact network (typically 256-512 hidden units) rather than solving optimization problems or computing inverse kinematics at each timestep. Most critically, the learned behaviors often surpass hand-engineered approaches in robustness, as the policy experiences diverse randomized conditions during training and develops strategies that generalize across parameter variations.

Key design innovations enable this end-to-end approach. Forward-biased goal sampling places navigation targets ahead of the robot rather than uniformly around it, encouraging forward locomotion rather than stationary pivoting. Heading-aware velocity scaling adjusts commanded forward velocity based on angular alignment with the goal: when facing the target, maximum forward speed is permitted; when misaligned, rotational correction takes priority. Distance-adaptive speed control increases velocity as goals approach, enabling both long-range traversal and precise terminal positioning. Graduated reward shaping balances competing objectives—goal achievement, locomotion smoothness, energy efficiency, and stability—through carefully weighted terms that guide policy learning without over-constraining exploration.

This methodology extends the Humanoid-Gym framework [3], which demonstrated velocity-tracking locomotion but not goal-directed navigation. By transforming low-level joint control from velocity tracking to spatial navigation, this work enables autonomous

robot operation suitable for real-world tasks: delivering objects to specified locations, patrolling designated waypoints, or approaching users who request assistance. The approach remains accessible—training completes in 3-4 hours on a single NVIDIA RTX 4080 GPU—while providing capabilities previously requiring hierarchical architectures or expensive motion planning infrastructure.

Figure 2 illustrates this architectural distinction, contrasting traditional hierarchical decomposition with the end-to-end approach adopted in this work.

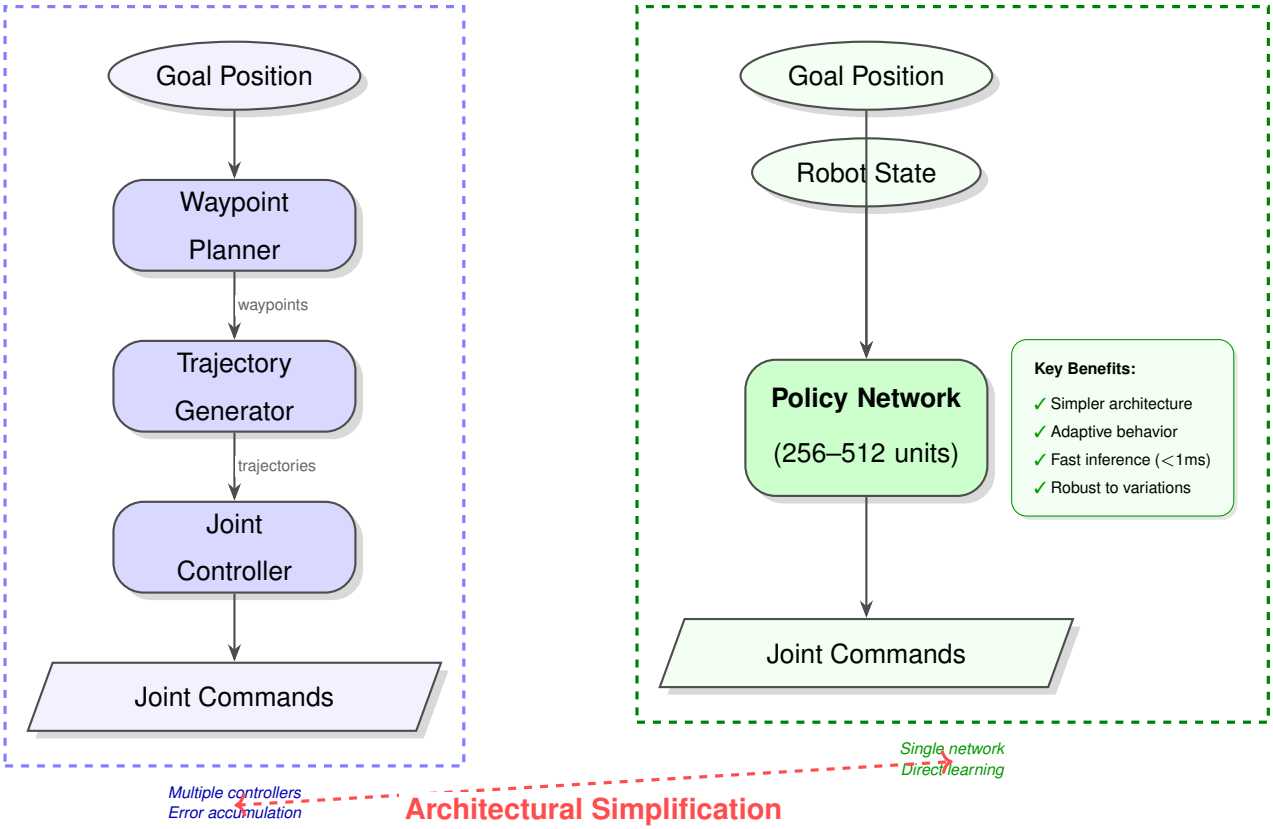


Figure 2: Comparison of traditional hierarchical navigation architecture versus the end-to-end approach adopted in this work. The hierarchical architecture requires multiple coordinated controllers and intermediate representations (waypoints, trajectories), accumulating errors and computational costs. The end-to-end approach directly maps goal positions and robot state to joint commands through a single learned policy, eliminating abstraction layers while achieving superior robustness through diverse training experiences.

3.6.2 Communication-Realistic Validation Through Authentic ROS 2 Middleware

With a navigation policy trained through domain randomization, existing methodologies proceed directly to physical deployment [3]. The policy has experienced physics variations—randomized masses, friction coefficients, joint damping—but has never encountered communication constraints present in real robotic systems. During training and pure-simulation validation, the policy receives observations through direct Python function calls with nanosecond latencies. During physical deployment, observations flow through ROS 2 topics, with serialization overheads, network transport latencies (1-10ms), and asynchronous publishing creating temporal misalignment between sensor measurements and control decisions.

This communication gap could degrade performance or introduce instability. Consider a humanoid maintaining balance: if expected joint state observations arrive 5ms late due to middleware latency, the policy's learned control strategy—tuned for instantaneous feedback during training—might generate inappropriate commands, destabilizing the robot. Policies validated only in pure simulation have no opportunity to develop robustness to these timing characteristics before encountering them on expensive hardware.

This thesis introduces simulation-to-simulation validation employing authentic ROS 2 Humble communication protocols as an intermediate verification step. A second simulation environment implements the complete middleware stack: Unitree-compatible topics (`/lowstate` for proprioceptive sensing, `/lowcmd` for motor commands, `/utlidar/cloud` for LiDAR perception), realistic publication frequencies (50 Hz for joint states, 10 Hz for point clouds), message serialization through standard ROS 2 types (`sensor_msgs/JointState`, `nav_msgs/Odometry`), and DDS transport introducing characteristic latencies and asynchronous delivery patterns.

Policies trained in Isaac Lab deploy to this communication-realistic environment before physical hardware testing. If the policy maintains stable locomotion and achieves navigation goals under ROS 2 constraints, confidence in hardware deployment increases substantially. If communication latencies cause instability or degraded performance, these issues surface in simulation where debugging incurs no hardware damage risks.

This staged validation—physics-based training with domain randomization, followed by communication-realistic evaluation with authentic middleware—provides two verification stages before requesting precious physical robot time.

The framework replicates communication constraints present in real Unitree robots. The `/lowstate` topic publishes 29-DOF joint positions and velocities, alog 6-DOF IMU data, matching Unitree SDK2 message structures. Commands flow through `/lowcmd` topics using position-control or torque-control modes, respecting actuator bandwidth limitations. Temporal synchronization employs ROS 2 clock mechanisms (`/clock` topic) ensuring consistent timestamps across distributed nodes. Quality-of-service policies match real-world configurations: reliable delivery for control commands (cannot afford message loss), and best-effort for high-frequency sensor streams (latest measurement matters more than historical completeness).

This approach addresses the communication validation gap identified in existing sim-to-sim transfer research. While prior work validates policies across different physics engines [3], it typically ignores middleware realism, evaluating policies through direct simulator function calls. By introducing authentic ROS 2 communication as an intermediate validation layer, this work provides credible assessment of policy readiness before physical deployment—particularly valuable for educational contexts where hardware damage risks must be minimized.

3.6.3 Educational Accessibility Through Containerized Deployment

The validation framework proves most valuable when accessible to students with limited computational resources and no physical robot access. This thesis packages the complete system—Isaac Sim 4.5, Isaac Lab 2.2, ROS 2 Humble, CycloneDDS middleware, trained policy checkpoints, and all dependencies—into Docker containers with reproducible multi-stage builds. Students clone a repository, execute a single build command, and obtain a complete training and validation environment without manual dependency resolution or version conflicts.

This containerization strategy addresses educational access barriers identified in the literature. Transformer-based approaches [2] require massive GPU clusters, which are

unavailable to most students. Motion capture methods [13] demand specialized laboratory infrastructure. Iterative sim-to-real validation assumes ready hardware access for testing. In contrast, this framework enables students to develop, train, and validate locomotion policies entirely in simulation on personal workstations or shared university servers. They progress through physics-based training (requiring GPU resources but no hardware access) and communication-realistic validation (introducing ROS 2 constraints) before requesting physical robot time.

The educational benefits extend beyond hardware preservation. Students gain hands-on experience with industrial-standard tools: ROS 2 for robotics middleware, Docker for reproducible deployment, Isaac Lab for GPU-accelerated simulation, and PPO for policy optimization. They iterate rapidly—training a policy in 6-8 hours, evaluating in sim-to-sim validation immediately, then refining based on observed behaviors—without scheduling physical robot access or risking expensive hardware during exploratory experiments. When confident in policy performance through two simulation-based verification stages, they request robot time for final validation, maximizing learning opportunities while minimizing hardware risks.

This approach enables students to engage with humanoid robotics research using shared physical resources. A single university robot serves an entire graduate program: students develop policies independently in simulation, demonstrate successful sim-to-sim validation to instructors, then receive supervised robot access for final verification. This staged progression from pure simulation, through communication-realistic validation, to physical deployment balances educational accessibility with research rigor.

3.6.4 Positioning and Future Validation

This thesis adopts direct reinforcement learning for its accessibility, reproducibility, and proven transferability [3], but diverges from existing frameworks through staged validation and educational focus. The ultimate validation remains sim-to-real transfer to physical Unitree G1 hardware (Phase 7, future work beyond this thesis scope). However, by providing two simulation-based verification stages before physical deployment, this work enables safer progression toward hardware testing particularly suitable for educational

contexts where thousands of students must share limited resources.

Future work will assess whether communication-realistic sim-to-sim validation improves first-deployment success rates and reduces hardware damage incidents compared to direct sim-to-real approaches. Preliminary sim-to-sim results demonstrate successful policy operation under authentic ROS 2 constraints, providing encouraging evidence for eventual physical transfer while maintaining the educational accessibility that motivated this research. The framework positions simulation-to-simulation validation not as a replacement for physical deployment but as a credible intermediate step that increases deployment confidence and reduces iteration costs for resource-constrained educational institutions.

4 Problem Formulation

This section formalizes the humanoid locomotion control problem as a Markov Decision Process (MDP), providing the mathematical foundation for the reinforcement learning approach developed in subsequent chapters. The formal specification enables precise discussion of state representations, action spaces, reward structures, and optimization objectives.

4.1 Markov Decision Process Formulation

The goal-directed locomotion task is formulated as a finite-horizon MDP defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma, T)$, where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ the action space, P the transition dynamics, R the reward function, γ the discount factor, and T the episode horizon.

4.1.1 State Space $\mathcal{S}$

The state space $\mathcal{S} \subseteq \mathbb{R}^{n_s}$ comprises proprioceptive observations available to the policy. For the base locomotion task, $n_s = 125$:

$$\mathbf{s}_t = \left[\mathbf{p}_{goal}^{base}, \mathbf{v}_{base}, \boldsymbol{\omega}_{base}, \mathbf{g}_{proj}, \mathbf{q}, \dot{\mathbf{q}}, \mathbf{v}_{cmd}, \mathbf{a}_{t-1}\right]^\top \tag{1}$$

Table 2 details each component of the state vector.

Table 2: State space components for base locomotion task

Component	Dimension	Description
$\mathbf{p}_{goal}^{base}$	2	Goal position in base frame (x, y)
$\mathbf{v}_{base}$	3	Base linear velocity (v_x, v_y, v_z)
$\boldsymbol{\omega}_{base}$	3	Base angular velocity $(\omega_x, \omega_y, \omega_z)$
$\mathbf{g}_{proj}$	3	Projected gravity vector (orientation encoding)
$\mathbf{q}$	29	Joint positions (radians)
$\dot{\mathbf{q}}$	29	Joint velocities (rad/s)
$\mathbf{v}_{cmd}$	3	Velocity commands $(v_x^{cmd}, v_y^{cmd}, \omega_z^{cmd})$
$\mathbf{a}_{t-1}$	29	Previous actions
Total	**125**	

For the social navigation variant, the state space extends to $\mathcal{S} \subseteq \mathbb{R}^{141}$ with additional LiDAR sector observations $\mathbf{l} \in \mathbb{R}^{16}$:

$$\mathbf{s}_t^{social} = [\mathbf{s}_t, \mathbf{l}_t]^\top, \quad l_i = \frac{\min(d_i, d_{max})}{d_{max}} \in [0, 1] \tag{2}$$

where l_i represents the normalized minimum distance to any obstacle within the i-th angular sector (each spanning $22.5\circ$), and $d_{max} = 10$ m is the maximum LiDAR range. The state representation uses proprioceptive observations rather than privileged simulator state, ensuring policies can execute on physical hardware.

4.1.2 Action Space $\mathcal{A}$

The action space $\mathcal{A} \subseteq \mathbb{R}^{29}$ consists of target joint position offsets from a default standing pose $\mathbf{q}_{default}$:

$$\mathbf{a}_t = \Delta\mathbf{q}_{target} \in [-1, 1]^{29} \tag{3}$$

Actions are scaled by joint-specific position limits and converted to target positions:

$$q_{target,j} = q_{default,j} + a_j \cdot \Delta q_{max,j} \tag{4}$$

where $\Delta q_{max,j}$ defines the maximum position deviation for joint j. A proportional-derivative (PD) controller converts position targets to joint torques:

$$\tau_j = K_p(q_{target,j} - q_j) + K_d(\dot{q}_{target,j} - \dot{q}_j) \tag{5}$$

with gains $K_p = 50$ N·m/rad and $K_d = 1$ N·m·s/rad, matching Unitree G1 default parameters. Position-based actions with PD control match the interface available on physical Unitree robots and provide implicit compliance for contact stability.

4.1.3 Transition Dynamics P

The transition function $P : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ maps state-action pairs to probability distributions over next states. The dynamics are defined implicitly through physics simulation:

$$\mathbf{s}_{t+1} \sim P(\cdot|\mathbf{s}_t, \mathbf{a}_t) = \text{PhysX}(\mathbf{s}_t, \mathbf{a}_t, \boldsymbol{\xi}_t) \tag{6}$$

where PhysX denotes the NVIDIA PhysX 5 simulation engine, and $\boldsymbol{\xi}_t$ represents stochastic elements introduced through domain randomization. Table 3 specifies the randomization distributions.

Table 3: Domain randomization parameters

Parameter	**Distribution**	**Rationale**
Link masses	$\mathcal{U}[0.85m, 1.15m]$	Manufacturing tolerances, payload variation
Ground friction	$\mathcal{U}[0.5, 1.2]$	Surface material variation
Joint damping	$\mathcal{U}[0.8d, 1.2d]$	Actuator wear, temperature effects
Motor strength	$\mathcal{U}[0.9\tau, 1.1\tau]$	Battery voltage, motor efficiency
Observation noise	$\mathcal{N}(0, \sigma_q^2)$, $\sigma_q = 0.01$ rad	Encoder noise
Action delay	$\mathcal{U}\{1, 2, 3\}$ steps	Communication latency

These ranges derive from common practice in legged robotics [3, 17] and approximate expected hardware variation.

4.1.4 Reward Function R

The reward function $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is a weighted combination of task objectives and regularization terms:

$$R(\mathbf{s}_t, \mathbf{a}_t) = \sum_{i=1}^{N} w_i \cdot r_i(\mathbf{s}_t, \mathbf{a}_t) \tag{7}$$

Table 4 provides the complete reward specification, including mathematical definitions for the base locomotion task.

Table 4: Reward function specification for base locomotion

Term	**Weight**	**Mathematical Definition**
Task Objectives		
Linear velocity tracking	1.0	$r_{vel} = \exp\left(-\|\mathbf{v}_{xy} - \mathbf{v}_{xy}^{cmd}\|^2/\sigma_{vel}^2\right)$, $\sigma_{vel} = 0.25$
Angular velocity tracking	1.0	$r_{\omega} = \exp\left(-(\omega_z - \omega_z^{cmd})^2/\sigma_{\omega}^2\right)$, $\sigma_{\omega} = 0.25$
Goal distance (coarse)	0.3	$r_{goal,c} = \exp\left(-d_{goal}^2/\sigma_c^2\right)$, $\sigma_c = 5.0$
Goal distance (fine)	0.2	$r_{goal,f} = \exp\left(-d_{goal}^2/\sigma_f^2\right)$, $\sigma_f = 3.0$
Heading alignment	0.3	$r_{head} = \exp\left(-\theta_{error}^2/\sigma_h^2\right)$, $\sigma_h = 0.5$
Feet air time	0.75	$r_{air} = \sum_{i \in feet} \max(0, t_{air,i} - 0.1)$
Regularization (Penalties)		
Action rate	-0.005	$r_{act} = -\|\mathbf{a}_t - \mathbf{a}_{t-1}\|^2$
Joint deviation (arms)	-0.1	$r_{joint} = -\|\mathbf{q}_{arms} - \mathbf{q}_{default,arms}\|^2$
Feet slide	-0.1	$r_{slide} = -\sum_{i \in feet} \|\mathbf{v}_{foot,i}\| \cdot c_i$
Flat orientation	-1.0	$r_{flat} = -\|\mathbf{g}_{proj,xy}\|^2$
Backwards velocity	-0.5	$r_{back} = -\mathbb{1}[v_x < -0.1]$
Termination	-200	$r_{term} = -\mathbb{1}[\text{fall}]$

The exponential kernel form $\exp(-x^2/\sigma^2)$ provides smooth gradients for optimization. The dual goal rewards address sparsity: coarse ($\sigma = 5.0$) provides long-range gradient, fine ($\sigma = 3.0$) enables precise positioning.

4.1.5 Discount Factor and Episode Horizon

The discount factor $\gamma = 0.99$ balances immediate and future rewards, with effective horizon $\frac{1}{1-\gamma} = 100$ steps. Episodes terminate after $T = 1000$ steps (20 seconds at 50 Hz control frequency) or upon early termination conditions:

- **Base contact:** Any non-foot body part contacts the ground (fall detection)
- **Height violation:** Base height falls below threshold ($z_{base} < 0.3$ m)
- **Orientation violation:** Excessive body tilt ($|\theta_{roll}| > 1.0$ rad or $|\theta_{pitch}| > 1.0$ rad)

Early termination triggers the termination penalty ($r_{term} = -200$), providing strong gradient against unsafe states.

4.2 Optimization Objective

The objective is to find a policy $\pi_\theta : \mathcal{S} \to \Delta(\mathcal{A})$ parameterized by neural network weights θ that maximizes expected cumulative discounted reward:

$$\pi^* = \arg\max_{\pi_\theta} J(\theta) = \arg\max_{\pi_\theta} \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} \gamma^t R(\mathbf{s}_t, \mathbf{a}_t) \right] \tag{8}$$

where $\tau = (\mathbf{s}_0, \mathbf{a}_0, r_0, \mathbf{s}_1, \mathbf{a}_1, r_1, \ldots)$ denotes a trajectory sampled under policy π_θ, with initial states drawn from distribution $\mathbf{s}_0 \sim \rho_0$.

The policy $\pi_\theta(\mathbf{a}|\mathbf{s})$ is represented as a diagonal Gaussian distribution:

$$\pi_\theta(\mathbf{a}|\mathbf{s}) = \mathcal{N}(\boldsymbol{\mu}_\theta(\mathbf{s}), \text{diag}(\boldsymbol{\sigma}_\theta^2)) \tag{9}$$

where the mean $\boldsymbol{\mu}_\theta(\mathbf{s}) \in \mathbb{R}^{29}$ is output by the actor network, and the standard deviation $\boldsymbol{\sigma}_\theta \in \mathbb{R}^{29}$ is learned as state-independent parameters. During training, actions are sampled from this distribution to enable exploration; during evaluation, the deterministic mean $\boldsymbol{\mu}_\theta(\mathbf{s})$ is typically used.

4.3 Problem Characteristics and Algorithm Selection

The MDP exhibits characteristics that motivate algorithm selection: high-dimensional continuous state ($\mathbb{R}^{125}$) and action ($\mathbb{R}^{29}$) spaces requiring function approximation; partial observability (proprioceptive observations only); stochastic dynamics from domain randomization; and real-time execution constraints (50 Hz, requiring sub-millisecond inference).

Proximal Policy Optimization (PPO) [18] was selected based on: (1) stability through trust region constraints, critical for safety-sensitive robotics; (2) proven sim-to-real transfer on Unitree platforms [3] and ANYmal [17]; (3) effective parallelization enabling 100,000+ transitions/second with 512 environments; and (4) mature RSL-RL implementation optimized for Isaac Lab. Alternative algorithms (SAC, TD3) were not evaluated; comparative studies remain for future work.

5 Methodology

This thesis employs a phased methodology that progresses from environment creation through policy training to communication-realistic validation. The approach addresses three interconnected challenges: creating a high-fidelity simulation environment that accurately represents the physical deployment context, training locomotion policies capable of goal-directed navigation through low-level joint control, and validating those policies under realistic communication constraints before physical deployment. Figure 3 illustrates the complete methodological pipeline, showing data flow between phases and the progressive refinement from pure simulation toward deployment readiness.

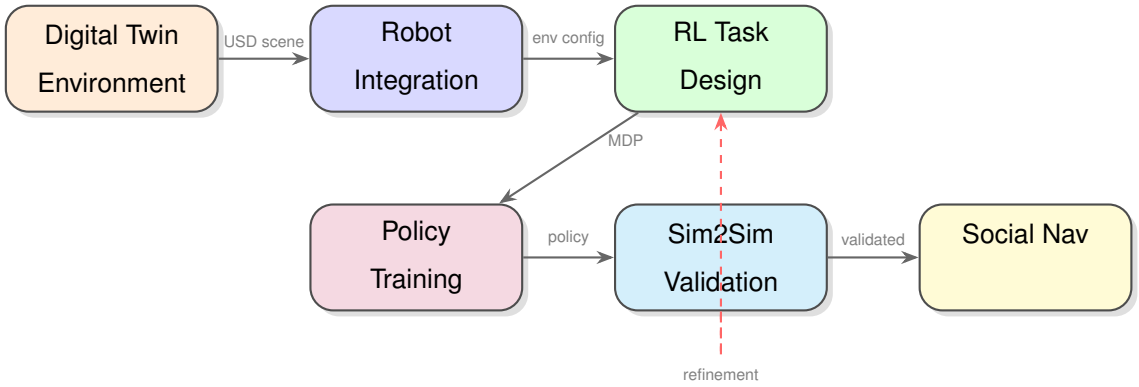


Figure 3: Methodological pipeline showing the six development phases. The dashed feedback loop indicates iterative refinement based on sim2sim validation results.

The methodology comprises six phases: (1) Digital Twin Environment Creation, converting the physical Laboratorio Colivri from Unity format to simulation-ready USD assets; (2) Robot Integration, configuring the Unitree G1 within the Isaac Lab framework; (3) Custom RL Task Design, implementing goal-directed navigation through novel command generators and reward functions; (4) Policy Training, applying PPO-based reinforcement learning across 512 parallel environments; (5) Sim2Sim ROS 2 Validation, evaluating trained policies under authentic communication constraints; and (6) Social Navigation Infrastructure, establishing LiDAR-based obstacle avoidance foundations for future work. The following subsections detail each phase, specifying design decisions,

implementation strategies, and integration requirements.

5.1 Digital Twin Environment Creation

The first phase addresses a fundamental requirement for credible simulation: accurately representing the physical environment where the robot will operate. The Laboratorio Colivri at Universidad de los Andes serves as the deployment context, requiring conversion of an existing Unity-based virtual environment to NVIDIA's Universal Scene Description (USD) format compatible with Isaac Sim. This conversion process involved three major stages: format transformation, material reconstruction, and physics configuration.

5.1.1 Unity to USD Format Conversion

The original Laboratorio Colivri environment was as a Unity project containing architectural models, furniture, equipment, and decorative elements representing the physical laboratory space. Converting this environment to USD format required an intermediate export-import pipeline, as direct Unity-to-USD conversion tools did not preserve all necessary scene information.

The conversion pipeline employed FBX (Filmbox) as an intermediate format. Unity's FBX exporter preserved mesh geometry, UV mappings, and hierarchical scene structure, producing individual FBX files for scene components. These files were subsequently imported into NVIDIA Omniverse using the built-in FBX importer, which generated USD representations maintaining geometric fidelity. The process yielded 388 individual prop USD files representing scene objects (furniture, equipment, structural elements) plus a main building model totaling approximately 91 MB of scene data.

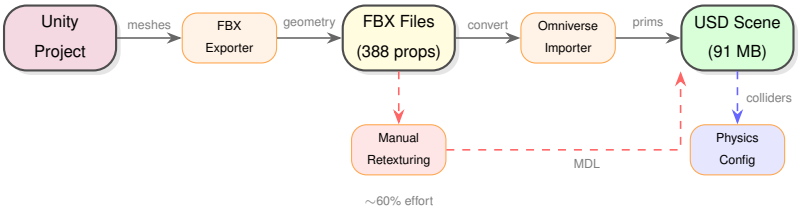


Figure 4: Unity to USD conversion pipeline. The process employed FBX as an intermediate format, preserving mesh geometry and UV mappings. Manual retexturing (approximately 60% of total effort) reconstructed materials using Omniverse MDL, followed by physics configuration for collision geometry.

5.1.2 Material Reconstruction and Retexturing

The format conversion introduced significant material fidelity challenges. Unity's material system (Standard, URP, or HDRP shaders) differs fundamentally from Omniverse's MDL (Material Definition Language) materials. During FBX export, material references were preserved but texture paths became invalid, and shader parameters required manual translation to MDL equivalents.

Material reconstruction proceeded systematically across all 388 scene props. For each object, the process involved: (1) identifying original texture assets from the Unity project, (2) reimporting textures into Omniverse with correct color space settings (sRGB for albedo/diffuse, linear for normal maps and roughness), (3) creating MDL materials with appropriate parameters (base color, roughness, metallic, normal mapping), and (4) applying materials to mesh surfaces with correct UV coordinate mappings. Table 5 summarizes the material property mappings between Unity Standard shader and Omniverse MDL.

Table 5: Material Property Mapping: Unity Standard Shader to Omniverse MDL

Unity Property	**MDL Property**	**Conversion Notes**
Albedo (_MainTex)	diffuse_color_texture	Direct mapping, sRGB color space
Metallic (_Metallic)	metallic_constant	Value range [0,1] preserved
Smoothness	roughness	Inverted: roughness = 1 - smoothness
Normal Map (_BumpMap)	normal_texture	Linear color space, tangent space
Occlusion (_OcclusionMap)	ao_texture	Direct mapping, linear space
Emission (_EmissionMap)	emissive_color_texture	Requires intensity scaling

The retexturing process accounted for approximately 60% of total environment creation time, reflecting the labor-intensive nature of material reconstruction when crossing between incompatible material systems. However, this investment ensures visual fidelity between the digital twin and physical laboratory, enabling realistic sensor simulation (particularly for camera-based perception) and accurate lighting interactions.

5.1.3 Physics Configuration

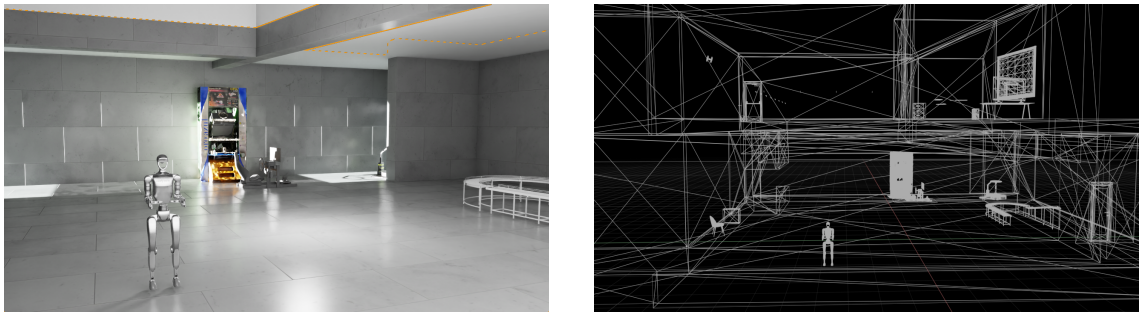
Accurate physics simulation requires more than visual fidelity; collision geometry, surface properties, and contact behaviors must reflect physical reality for meaningful locomotion training. The physics configuration phase addressed three requirements: collision mesh generation, physics material assignment, and ground plane configuration.

Collision Mesh Generation: Visual meshes optimized for rendering often contain excessive polygon counts and non-manifold geometry unsuitable for physics simulation. For each scene object, collision approximations were generated using convex decomposition (for complex shapes) or primitive fitting (boxes, capsules, spheres for simple shapes). Isaac Sim's collision mesh generation tools automated much of this process, though manual refinement was required for objects with significant concavities or thin features.

Physics Material Assignment: Contact dynamics depend on surface material properties including friction coefficients and restitution (bounciness). Physics materials were assigned based on real-world surface types: concrete floors received high static friction

($\mu_s = 0.8$) and moderate dynamic friction ($\mu_d = 0.6$); metal surfaces received lower friction values; and rubber foot contacts were configured to match Unitree G1 specifications. These parameters directly influence locomotion stability and policy behavior.

Ground Plane Configuration: The Laboratorio Colivri features a raised floor platform where the G1 robot operates. The ground plane was set to $z = 1.2$ m elevation to match physical laboratory conditions, with robot spawn positions adjusted accordingly. This configuration ensures that trained policies exhibit correct contact dynamics and fall recovery behaviors relative to the actual deployment surface.



(a) Rendered view (b) Wireframe view

Figure 5: The completed Laboratorio Colivri digital twin in NVIDIA Isaac Sim with the Unitree G1 humanoid robot: (a) rendered view demonstrating visual fidelity after material reconstruction, including textured walls, floor tiles, equipment, and lighting; (b) wireframe view revealing the underlying collision geometry used for physics simulation, showing mesh structures for environment props and the robot model.

5.2 Robot Integration and Isaac Lab Extension

With the environment prepared, the second phase integrates the Unitree G1 humanoid robot into the Isaac Lab framework, establishing the simulation infrastructure for reinforcement learning. This phase involves configuring the robot model, creating an Isaac Lab extension for project-specific customizations, and registering gymnasium-compatible training environments.

5.2.1 Unitree G1 Robot Configuration

The Unitree G1 humanoid robot provides 29 actuated degrees of freedom distributed across lower body (12 DOF: hip, knee, ankle), upper body (14 DOF: shoulder, elbow, wrist), and torso (3 DOF: waist). Figure 6 illustrates the robot's hardware architecture, showing the distribution of joint motors, sensors, and structural components. Table 6 details the joint distribution and range limits.

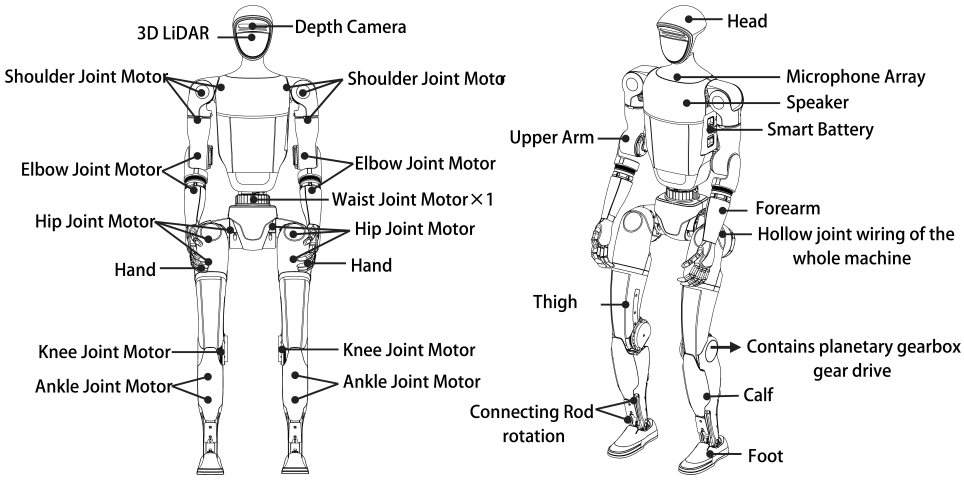


Figure 6: Unitree G1 humanoid robot hardware component overview showing the distribution of joint motors across the kinematic chain (left: front view, right: side view). The platform integrates 3D LiDAR and depth camera sensors in the head unit, with actuators distributed across shoulder, elbow, hip, knee, and ankle joints. Adapted from Quadruped.de [19].

Table 6: Unitree G1 Joint Configuration (29 DOF)

Body Part	**Joints per Side**	**Total DOF**	**Joint Types**
Hip	3	6	Roll, pitch, yaw
Knee	1	2	Pitch
Ankle	2	4	Pitch, roll
Waist	3 (central)	3	Roll, pitch, yaw
Shoulder	3	6	Roll, pitch, yaw
Elbow	1	2	Pitch
Wrist	1	2	Roll
Gripper	2	4	Finger joints (fixed in locomotion)
Total Actuated	—	**29**	—

The robot configuration inherits from Isaac Lab's proven `G1FlatEnvCfg` base class, which provides validated motor parameters, joint limits, and default poses established through prior sim-to-real transfer work [3]. This inheritance strategy leverages existing validation while enabling project-specific customizations for goal-directed navigation.

5.2.2 Isaac Lab Extension Architecture

The project implements a standalone Isaac Lab extension (`digital_twin_colivri_g1`) following NVIDIA's recommended extension pattern. This architecture enables custom environment definitions, reward functions, and command generators while maintaining compatibility with Isaac Lab's core infrastructure. Figure 7 illustrates the extension structure, and Table 7 describes the key components.

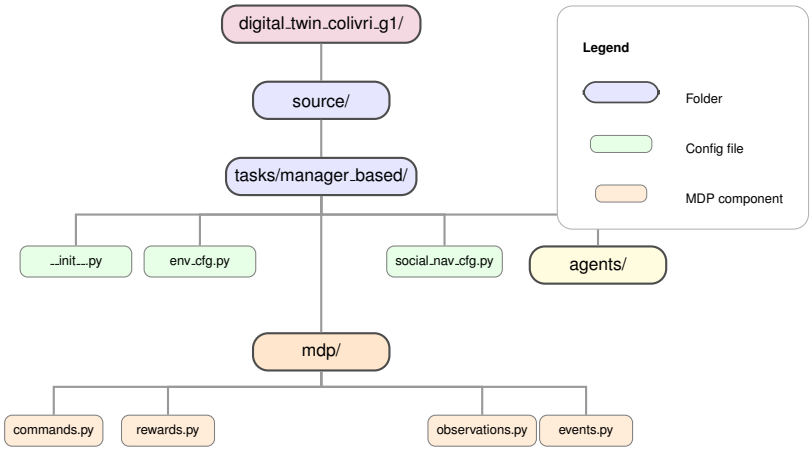


Figure 7: Isaac Lab extension architecture for the digital twin project. The extension follows NVIDIA's recommended pattern with environment configurations at the task level and custom MDP components (commands, rewards, observations, events) in a dedicated subdirectory.

Table 7: Extension Component Descriptions

Component	**Type**	**Description**
`__init__.py`	Config	Gymnasium task registration and entry points
`env_cfg.py`	Config	Base environment configuration (512 envs, rewards, commands)
`social_nav_cfg.py`	Config	Social navigation variant with LiDAR and obstacles
`agents/`	Folder	PPO hyperparameter configurations (RSL-RL, SKRL)
`commands.py`	MDP	Goal position and velocity command generators (570 LOC)
`rewards.py`	MDP	Navigation reward functions (409 LOC)
`observations.py`	MDP	State observations including LiDAR sectors (91 LOC)
`events.py`	MDP	Environment events and obstacle spawning (162 LOC)

The extension registers four gymnasium-compatible tasks, summarized in Table 8:

Table 8: Registered Gymnasium Tasks

Task ID	Envs	Description
`Isaac-Colivri-G1-v0`	512	Training with goal-directed navigation and domain randomization
`Isaac-Colivri-G1-Play-v0`	10	Evaluation with disabled randomization for visualization
`Isaac-Colivri-G1-SocialNav-v0`	128	Social navigation training with LiDAR and obstacle avoidance
`Isaac-Colivri-G1-SocialNav-Play-v0`	4	Social navigation evaluation with sensor visualization

5.3 Custom RL Task Design for Goal-Directed Navigation

The third phase implements the core contribution of this work: a custom reinforcement learning task that enables point-to-point navigation through direct low-level joint control. Unlike hierarchical approaches that require separate trajectory planners and joint controllers, this task trains a single policy network to map goal positions and robot state directly to 29-DOF joint commands. The implementation comprises three interconnected components: goal command generation, goal-directed velocity commands, and custom reward functions.

5.3.1 Goal Command Generation

Navigation tasks require the robot to reach specified goal positions. The `SimpleGoalCommand` class generates goal positions and resamples goals throughout training episodes. Key design decisions address distribution shift and behavioral stability:

Forward-Biased Goal Sampling: Random uniform goal sampling often places targets behind the robot, requiring 180-degree rotations before forward progress can be made. This creates training instability and encourages stationary pivoting rather than efficient locomotion. The implementation biases goal sampling toward the robot's forward direction: at episode initialization, 80% of goals spawn within a 30-degree forward cone, with the remaining 20% in the forward hemisphere ($\pm$90 degrees). After reaching

a goal, subsequent goals use 65% forward bias to maintain locomotion momentum while introducing navigational variety.

Reach-Based Resampling: Traditional approaches resample goals at fixed time intervals regardless of task progress. This work employs reach-based resampling: new goals generate only when the robot achieves proximity to the current target (distance < 0.3 m). This approach ensures the policy receives consistent training signal for goal approach behaviors rather than arbitrary goal transitions.

Smooth Goal Transitions: Instantaneous goal changes can destabilize locomotion as velocity commands shift abruptly. The implementation interpolates between consecutive goals over 1.5 seconds, providing smooth command transitions that maintain balance during navigation.

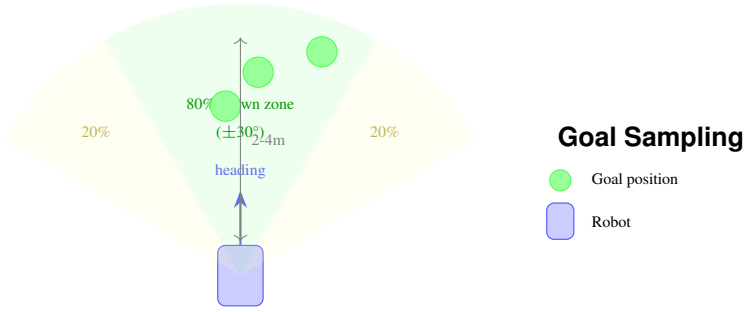


Figure 8: Forward-biased goal sampling distribution. At episode initialization, 80% of goals spawn in a narrow forward cone ($\pm 30°$) with 20% in the extended forward hemisphere. This bias encourages forward locomotion development while ensuring navigational coverage. Goals spawn at 2-4 meter distances from the robot.

5.3.2 Goal-Directed Velocity Commands

While goal positions specify navigation targets, the locomotion policy operates on velocity commands (linear velocity v_x, v_y and angular velocity ω_z). The `GoalDirectedVelocityComm` class transforms goal positions into velocity commands that guide the robot toward targets. This transformation incorporates three scaling mechanisms:

Distance-Based Speed Scaling: Velocity magnitude scales with distance to goal. When far from the target (> 3 m), full forward velocity is commanded. As distance

decreases, velocity reduces proportionally to enable precise terminal positioning:

$$v_{scale} = v_{min} + (1 - v_{min}) \cdot \min\left(1, \frac{d_{goal}}{d_{threshold}}\right) \tag{10}$$

where $v_{min} = 0.2$ (20% minimum velocity), d_{goal} is current distance to goal, and $d_{threshold} = 3.0$ m.

Heading-Aware Modulation: Forward velocity further scales based on angular alignment with the goal. When facing the target (heading error $< 15°$), full scaled velocity applies. When perpendicular or facing away, forward velocity reduces to prioritize rotational correction:

$$v_{heading} = 0.4 + 0.6 \cdot \cos(\theta_{error}) \tag{11}$$

where θ_{error} is the angle between robot's heading and the goal direction.

No Backwards Motion: The velocity command enforces $v_x \geq 0$, preventing backwards locomotion that often indicates policy confusion or local minima. The robot must rotate to face goals rather than walking backwards, promoting stable forward gait development.

Proportional Angular Control: Angular velocity commands apply proportional control toward the goal direction with gain $K_\omega = 5.0$:

$$\omega_z = K_\omega \cdot \theta_{error} \tag{12}$$

clamped to actuator limits (± 1.0 rad/s).

5.3.3 Custom Reward Functions

The reward function guides policy learning toward desired navigation behaviors while maintaining locomotion stability. The design balances multiple objectives through weighted reward terms, inheriting proven velocity-tracking rewards from the G1 base configuration while adding navigation-specific terms.

Dual Exponential Goal Distance Rewards: Two exponential terms provide multi-

scale goal approach incentives:

$$r_{coarse} = 0.3 \cdot \exp\left(-\frac{d_{goal}^2}{\sigma_{coarse}^2}\right), \quad \sigma_{coarse} = 5.0 \tag{13}$$

$$r_{fine} = 0.2 \cdot \exp\left(-\frac{d_{goal}^2}{\sigma_{fine}^2}\right), \quad \sigma_{fine} = 3.0 \tag{14}$$

The coarse term ($\sigma = 5.0$) provides gradient across the full navigation range, while the fine term ($\sigma = 3.0$) sharpens reward near the goal for precise positioning.

Heading Alignment Reward: Encourages the robot to face toward the goal:

$$r_{heading} = 0.3 \cdot \exp\left(-\frac{\theta_{error}^2}{\sigma_{heading}^2}\right), \quad \sigma_{heading} = 0.5 \tag{15}$$

Backwards Velocity Penalty: Discourages backwards motion that indicates policy confusion:

$$r_{backwards} = -0.5 \cdot \mathbb{1}[v_x < -0.1] \tag{16}$$

Reward Balance: The velocity tracking reward (weight 1.0) ensures stable locomotion, while navigation rewards (combined weight 0.8) provide directional guidance. This ratio emerged from empirical tuning—higher navigation weights destabilized learned gaits, while lower weights slowed goal convergence. The complete reward specification, including mathematical definitions is provided in Table 4 (Section 4.1.4).

The 125-dimensional observation space and domain randomization parameters are formally specified in Section 4.1 (Tables 2 and 3).

5.4 Policy Training

The fourth phase applies Proximal Policy Optimization (PPO) to train locomotion policies using the task configuration from Phase 3. Training leverages Isaac Lab's GPU-accelerated parallel simulation to achieve sample-efficient learning on a single workstation.

5.4.1 Training Configuration

Training employs 512 parallel environment instances, enabling collection of 512 state-action-reward tuples per simulation step. This parallelism accelerates training by approx-

imately 100× compared to sequential simulation, completing policy training in 3-4 hours on an NVIDIA RTX 4080 GPU.

The PPO configuration inherits from `G1FlatPPORunnerCfg`, which provides validated hyperparameters from prior sim-to-real transfer work:

Table 9: PPO Training Hyperparameters

Parameter	**Value**	**Description**
Learning rate	1×10^{-3}	Adam optimizer
Discount factor (γ)	0.99	Future reward weighting
GAE lambda (λ)	0.95	Advantage estimation
Clip range (ϵ)	0.2	Policy update constraint
Entropy coefficient	0.01	Exploration bonus
Value function coefficient	0.5	Critic loss weighting
Mini-batch size	4096	Samples per gradient update
Number of epochs	5	Passes over collected data
Network architecture	[512, 256, 128]	MLP hidden layers
Activation function	ELU	Nonlinearity

Figure 9 illustrates the actor-critic network architecture used in PPO training.

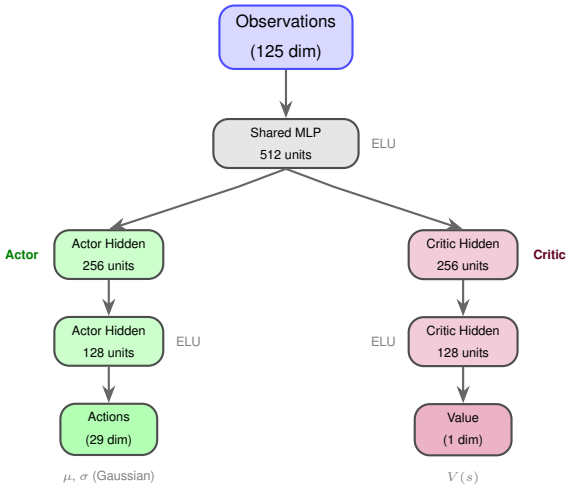


Figure 9: PPO actor-critic network architecture. The policy receives 125-dimensional observations and outputs 29-dimensional joint position targets through the actor branch, while the critic branch estimates state values for advantage computation. Both branches share initial feature extraction before specializing.

Design Rationale. Several configuration choices merit justification:

Parallel environments (512): The number of parallel environments balances GPU memory utilization against sample diversity. With 512 environments, each PPO iteration collects $512 \times 512 = 262{,}144$ samples, providing sufficient batch diversity for stable gradient estimation. Increasing to 1024+ environments exceeds available GPU memory (12GB on RTX 4080 Super) when combined with the 29-DOF robot model and ray-casting overhead. Decreasing the number of environments below 256 slows training disproportionately, as GPU utilization becomes suboptimal.

Network architecture [512, 256, 128]: The three-layer MLP architecture follows HumanoidGym's validated configuration, which achieved successful sim-to-real transfer on the same robot platform. The decreasing layer widths ($512 \rightarrow 256 \rightarrow 128$) progressively compress the 125-dimensional observation into a compact representation suitable for the 29-dimensional action output. Deeper or wider networks were not explored, as the inherited architecture proved sufficient for the task; network architecture ablations remain for future work.

Learning rate (10^{-3}) with adaptive schedule: The initial learning rate enables rapid

early progress, while the adaptive schedule (based on KL divergence monitoring) reduces the rate as training progresses, preventing oscillation near convergence. This schedule is implemented in RSL-RL and activates automatically when policy updates become too aggressive.

Entropy coefficient (0.01): The entropy bonus encourages exploration by penalizing deterministic policies. The value 0.01 maintains moderate exploration throughout training; higher values (0.05-0.1) caused unstable locomotion in preliminary experiments, while lower values (0.001) led to premature convergence to suboptimal gaits.

5.4.2 Domain Randomization

To promote sim-to-real transfer robustness, training applies domain randomization over physical parameters:

- **Mass variation:** $\pm 15\%$ randomization on link masses
- **Friction coefficients:** Ground friction uniformly sampled from [0.5, 1.2]
- **Joint damping:** $\pm 20\%$ variation on damping coefficients
- **Motor strength:** $\pm 10\%$ scaling on torque limits
- **Observation noise:** Gaussian noise on joint positions ($\sigma = 0.01$ rad) and velocities ($\sigma = 0.1$ rad/s)
- **Action delay:** Random 1-3 step delays on command execution

This randomization ensures policies experience diverse physical conditions during training, encouraging robust behaviors that generalize across parameter variations encountered during physical deployment.

Design Rationale for Randomization Ranges. The specific randomization ranges derive from common practice in legged robotics rather than measured hardware variation on the physical Unitree G1:

Mass variation ($\pm$15%): This range accounts for manufacturing tolerances (typically $\pm$5%), payload attachment (up to 10% body mass), and potential future modifications. The range follows recommendations from Hwangbo et al. [17] for quadruped locomotion.

Friction coefficients ([0.5, 1.2]): The range spans typical indoor surfaces (polished concrete $\mu \approx 0.5$, rubber flooring $\mu \approx 1.0$+). The upper bound exceeds typical measured values to provide a safety margin for sticky surfaces.

Joint damping ($\pm$20%) and motor strength ($\pm$10%): These ranges approximate expected actuator variation due to temperature, wear, and manufacturing tolerances. Actuator characterization data for Unitree G1 were not available; ranges follow conservative estimates from similar platforms.

Action delay (1-3 steps): At 50Hz control frequency, this corresponds to 20-60ms latency, covering expected communication and processing delays in the ROS2-based deployment pipeline. The sim2sim infrastructure introduces similar latencies, making this randomization particularly relevant.

Ideally, these ranges would derive from systematic hardware measurements. The current values represent conservative approximations that prioritize robustness over optimality; overly narrow ranges risk brittle policies that fail on real hardware, while overly wide ranges may unnecessarily constrain achievable performance.

5.4.3 Training Procedure

Training proceeds for approximately 2000-3000 iterations, with each iteration collecting 512 environment steps (262,144 total samples) and then performing 5 PPO update epochs. Checkpoints are saved every 100 iterations, enabling recovery from training failures and comparison across training stages.

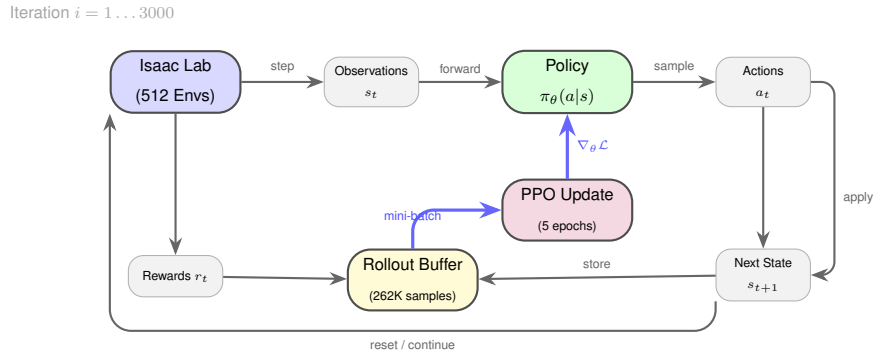


Figure 10: PPO training procedure. Each iteration collects 512 parallel environment steps and stores state-action-reward tuples in a rollout buffer. After accumulating 262,144 samples, PPO performs 5 update epochs on mini-batches, computing policy gradients $\nabla_\theta \mathcal{L}$ using the clipped surrogate objective. The loop continues for 2000-3000 iterations until convergence.

5.5 Simulation-to-Simulation ROS 2 Validation

The fifth phase implements the communication-realistic validation framework that distinguishes this work from direct sim-to-real approaches. Trained policies are evaluated in a second simulation environment that employs authentic ROS 2 Humble communication protocols, validating performance under realistic message passing constraints before physical deployment.

5.5.1 Validation Architecture

The sim2sim validation environment replicates communication patterns present in physical Unitree G1 deployments. Rather than invoking the policy through direct Python function calls (as during training), state observations and control commands flow through ROS 2 topics with realistic serialization, transport, and timing characteristics.

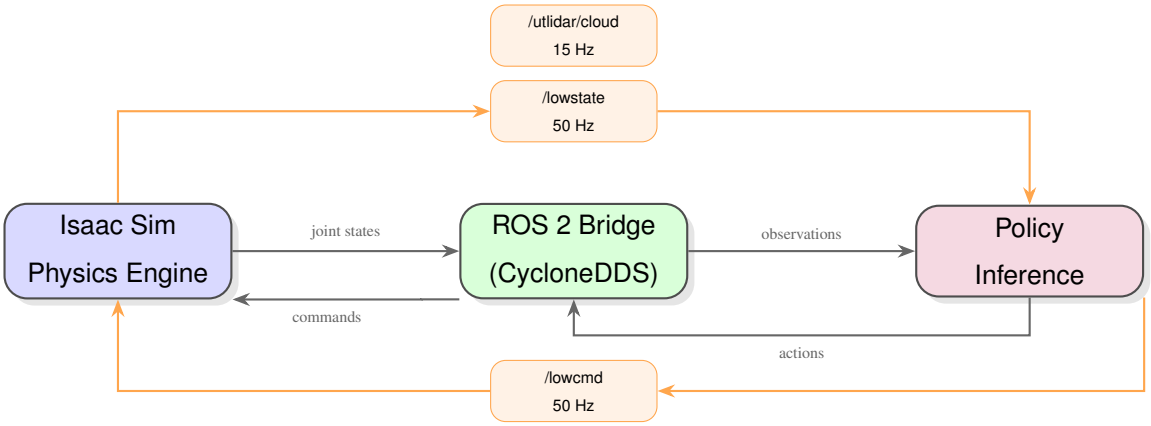


Figure 11: Sim2sim validation architecture. The ROS 2 bridge publishes robot state through Unitree-compatible topics at realistic frequencies. The policy node subscribes to observations and publishes commands, experiencing authentic communication latencies and asynchronous message delivery.

5.5.2 ROS 2 Bridge Implementation

The ROS 2 bridge implements a unified single-node architecture (`ColivriG1RobotDataManage` that combines all publishers and subscribers. This design eliminates threading complexity between separate publisher and subscriber nodes while maintaining Unitree SDK compatibility. Table 10 details the published topics:

Table 10: ROS 2 Topics Published by Sim2Sim Bridge

Topic	Message Type	Rate	Content
`/lowstate`	unitree_hg/LowState	50 Hz	29-DOF joint positions, velocities; IMU data
`/utlidar/cloud`	sensor_msgs/PointCloud2	15 Hz	RTX LiDAR point cloud
`/ground_truth/pose`	geometry_msgs/PoseStamped	50 Hz	World-frame robot pose
`/ground_truth/twist`	geometry_msgs/TwistStamped	50 Hz	Linear and angular velocity
`/colivri_g1/goal_marker`	visualization_msgs/Marker	1 Hz	Goal visualization (RViz)
`/clock`	rosgraph_msgs/Clock	Sim rate	Simulation time

The bridge subscribes to `/lowcmd` (unitree_hg/LowCmd) for motor commands, enabling external control integration and future teleoperation capabilities.

5.5.3 Communication Realism

Several mechanisms ensure communication-realistic validation:

Message Serialization: All data passes through ROS 2 message serialization/deserialization, introducing computational overhead absent from direct function calls. For complex messages like point clouds, this overhead can reach milliseconds.

DDS Transport: CycloneDDS middleware provides Data Distribution Service transport with configurable quality-of-service policies. The bridge uses reliable delivery for control commands (cannot afford message loss) and best-effort for high-frequency sensor streams (latest measurement matters most).

Asynchronous Publishing: Publishers and subscribers operate asynchronously. The policy node may receive observations at slightly different timestamps than the physics engine's actual state, introducing temporal misalignment characteristic of real systems.

Clock Synchronization: The `/clock` topic provides simulation time, enabling ROS 2 nodes to synchronize timestamps despite asynchronous execution.

5.5.4 Docker Containerization

The complete validation framework packages into Docker containers for reproducible deployment. A multi-stage Dockerfile (6 stages) constructs the environment:

1. **Base stage:** NVIDIA Isaac Sim 4.5.0 container
2. **ROS 2 stage:** Install ROS 2 Humble and CycloneDDS
3. **Dependencies stage:** Python packages and system libraries
4. **Messages stage:** Compile Unitree ROS 2 message definitions
5. **Extension stage:** Install Isaac Lab extension
6. **Final stage:** Configure entrypoint and volumes

Docker Compose orchestrates services and mounts volumes for trained checkpoints, logs, and configuration. X11 forwarding enables GUI visualization when needed.

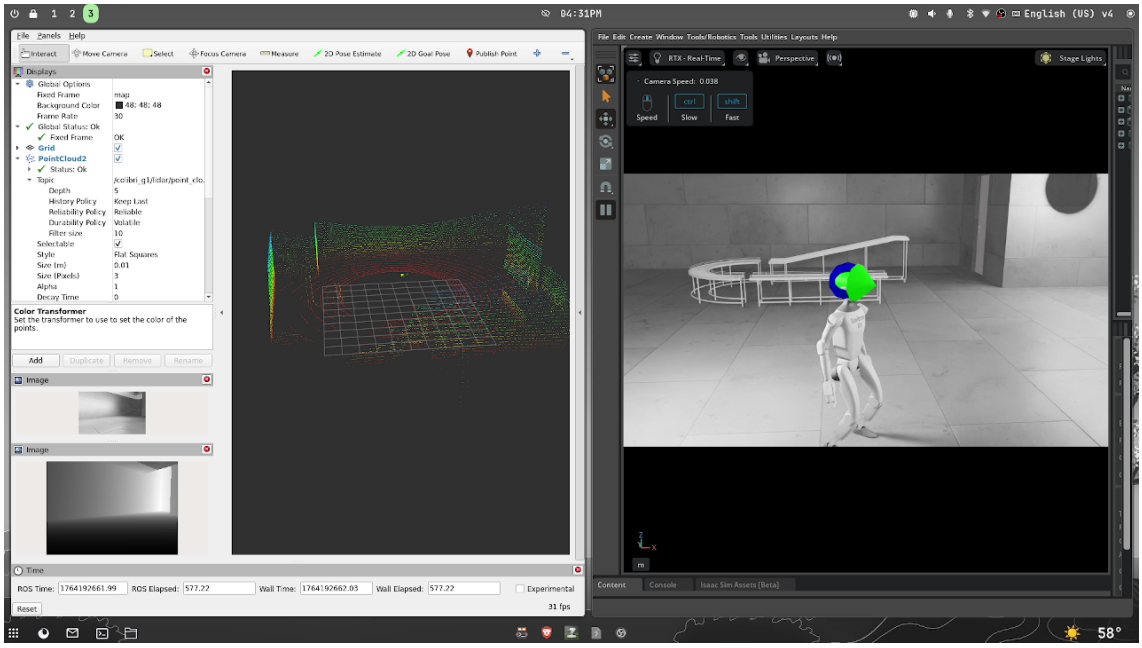


Figure 12: Sim2sim validation visualization in RViz2. The robot model, goal markers, and LiDAR point cloud display through standard ROS 2 visualization tools, enabling inspection of policy behavior under authentic communication constraints.

5.6 Social Navigation Infrastructure (Exploratory)

The sixth phase explores infrastructure for social navigation capabilities as preliminary groundwork for future research on obstacle avoidance and human-aware locomotion. This exploratory work implements the technical components—LiDAR integration, dynamic obstacles, and proximity-aware rewards—while documenting the fundamental challenges encountered that prevented effective policy convergence and inform future reward design improvements.

Figure 13 illustrates how the social navigation task extends the base locomotion framework by integrating LiDAR-based obstacle perception into the policy observation space.

5.6.1 LiDAR Sensor Integration

The social navigation task integrates a RayCaster-based LiDAR sensor mounted on the robot's torso link. Table 11 summarizes the sensor specifications.

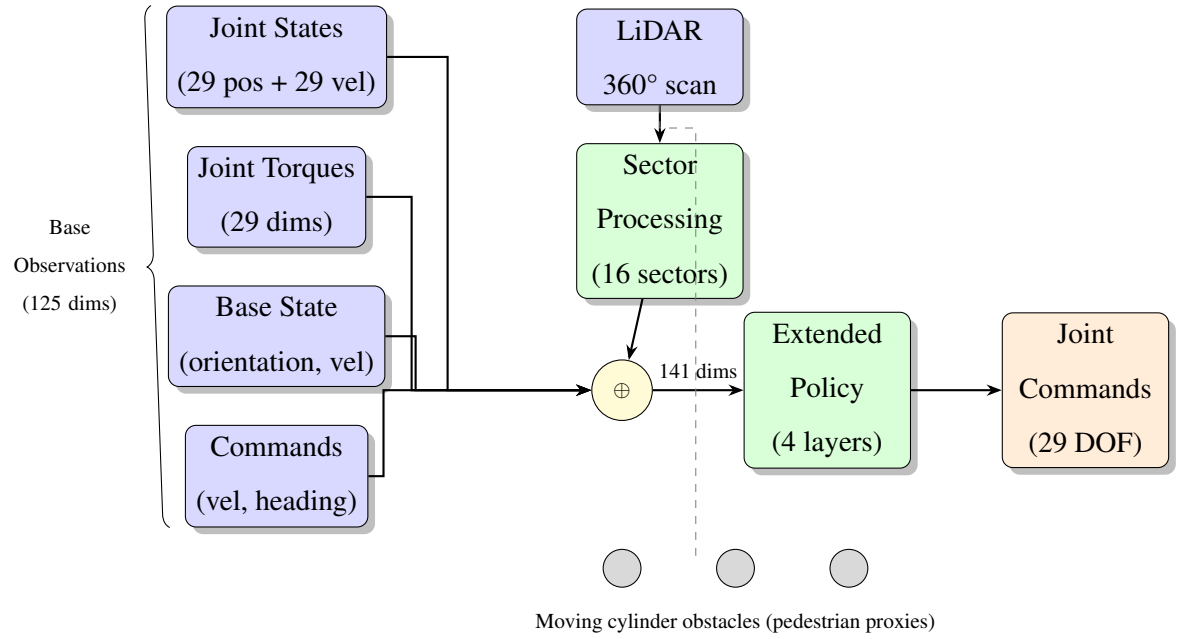


Figure 13: Social navigation task architecture extending base locomotion. LiDAR observations (16 sector minimum distances) are concatenated with proprioceptive state (125 dimensions) to form an extended 141-dimensional observation space. The policy network uses four hidden layers instead of three to accommodate the increased complexity.

Table 11: LiDAR sensor specifications for social navigation

Parameter	**Value**	**Notes**
Vertical channels	32	Per 360° sweep
Vertical FOV	$\pm 30°$	Total 60° coverage
Horizontal FOV	360°	Full surround view
Horizontal resolution	2°	180 rays per channel
Maximum range	10 m	Clipped for observations
Height filtering	0.2–2.0 m	Human-relevant heights
Mount location	torso_link	Stable during locomotion
Update rate	50 Hz	Synchronized with physics

For obstacle avoidance, raw point clouds are processed into sector-based minimum distances. The 360° field of view is divided into 16 angular sectors (22.5° each), and the minimum distance within each sector is extracted. This sector-based representation reduces the observation space from thousands of 3D points to a compact 16-dimensional vector suitable for policy input, while preserving directional obstacle information critical for avoidance behavior.

5.6.2 Dynamic Obstacle Infrastructure

Moving obstacles simulate pedestrians in the environment. The implementation uses dynamic rigid body cylinders approximating human body dimensions (0.3 m radius, 1.8 m height) with waypoint-based autonomous navigation. Table 12 details the obstacle configuration.

Table 12: Dynamic obstacle specifications

Parameter	**Value**	**Rationale**
Cylinder radius	0.3 m	Human body width
Cylinder height	1.8 m	Average human height
Cylinders per environment	3	Balance complexity/performance
Movement speed	0.3–0.5 m/s	Typical walking pace
Waypoint threshold	0.5 m	Trigger new waypoint
Linear damping	15.0	Predictable movement
Angular damping	15.0	Prevent rotation

A notable implementation detail involves the choice of dynamic rigid bodies over kinematic bodies. Isaac Sim 4.5/5.0 exhibits a visual update bug where kinematic bodies do not update their rendered positions when modified programmatically, despite correct physics state. The workaround employs dynamic rigid bodies with gravity disabled and high damping values, enabling position updates through velocity control while maintaining floor-level height. This approach ensures visual consistency between the rendered

scene and the underlying physics state, which is critical for both debugging and sim-to-real transfer validation.

5.6.3 Observation Space Extension

The social navigation task extends the base locomotion observation space with LiDAR-derived features. Table 13 summarizes the observation components.

Table 13: Social navigation observation space

Component	**Dimensions**	**Source**
Base velocity (linear/angular)	6	IMU
Joint positions	29	Encoders
Joint velocities	29	Encoders
Projected gravity	3	IMU orientation
Velocity commands	3	Command generator
Previous actions	29	Action buffer
Goal position (base frame)	3	World-to-base transform
Goal heading	1	Computed from goal position
Other proprioceptive	22	Various
Base observations	*125*	*Inherited from base task*
LiDAR sectors	16	RayCaster (normalized 0–1)
Total	**141**	

Each LiDAR sector observation represents the minimum distance to any obstacle within a 22.5° angular bin, normalized to the range $[0, 1]$ where 0 indicates an obstacle at the sensor and 1 indicates no obstacle within the 10 m maximum range. This normalization ensures consistent input scaling for the neural network policy.

5.6.4 Avoidance Reward Framework

The social navigation task extends the base reward structure with proximity-based penalties derived from LiDAR observations. Table 14 details the avoidance reward components.

Table 14: Social navigation reward components

Term	Weight	Threshold	Std	Description
Warning proximity	-0.5	1.5 m	0.5	Social distance violation
Personal space	-2.0	0.8 m	0.3	Critical proximity penalty
Smooth trajectory	$+0.1$	–	–	Action continuity reward

The proximity penalties use an exponential kernel:

$$r_{proximity} = -w \cdot \exp\left(-\frac{d_{min}^2}{\sigma^2}\right) \cdot \mathbb{1}[d_{min} < \tau] \tag{17}$$

where d_{min} is the minimum distance to any obstacle from LiDAR observations, w is the penalty weight, σ is the kernel standard deviation, and τ is the activation threshold. This formulation provides smooth gradients near obstacles rather than discrete penalties, facilitating gradient-based policy optimization. The smooth trajectory reward penalizes rapid action changes: $r_{smooth} = 0.1 \cdot \exp(-0.1 \cdot \|\Delta a\|_1)$, where Δa represents the action difference between consecutive timesteps.

Figure 14 illustrates the spatial penalty zones around obstacles, showing how the exponential kernel produces graduated penalties based on proximity.

5.6.5 PPO Configuration

The social navigation task requires modified Proximal Policy Optimization (PPO) hyperparameters compared to the base locomotion task due to the increased observation dimensionality and task complexity. Table 15 compares the configurations.

The deeper network architecture (four hidden layers instead of three) provides additional capacity for processing LiDAR sector observations alongside proprioceptive state.

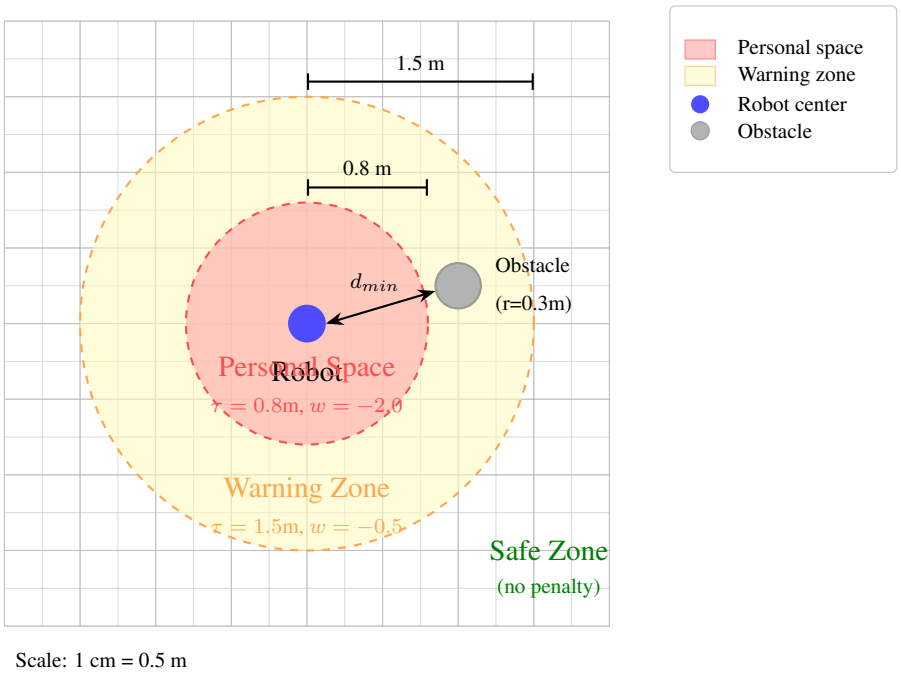


Figure 14: Top-down view of proximity penalty zones around obstacles. The personal space zone (0.8 m, red) triggers strong penalties ($w = -2.0$), while the warning zone (1.5 m, yellow) applies moderate penalties ($w = -0.5$). The exponential kernel provides smooth penalty gradients as the robot approaches obstacles, enabling gradient-based policy optimization.

Table 15: PPO configuration comparison: base locomotion vs. social navigation

Parameter	**Base Locomotion**	**Social Navigation**
Actor network	[512, 256, 128]	[512, 512, 256, 128]
Critic network	[512, 256, 128]	[512, 512, 256, 128]
Learning rate	1×10^{-3}	5×10^{-4}
Max iterations	1,500	2,000
Parallel environments	512	128
Entropy coefficient	0.005	0.01
Observation dimensions	125	141

The reduced learning rate and increased entropy coefficient promote more stable exploration in the higher-dimensional observation space. Fewer parallel environments (128 vs. 512) accommodate the increased per-environment memory requirements from LiDAR ray casting and cylinder collision computations.

6 Results and Discussion

This section presents the experimental results from training and evaluating reinforcement learning policies across two distinct task configurations:

1. **Isaac-Colivri-G1-v0** (Base Locomotion): Goal-directed velocity tracking with 125-dimensional observations, trained with 512 parallel environments over 1,500 iterations.

2. **Isaac-Colivri-G1-SocialNav-v0** (Social Navigation): Extended task with LiDAR-based obstacle awareness, 141-dimensional observations, trained with 128 parallel environments over 2,000 iterations.

The base locomotion task serves as the foundation, demonstrating stable bipedal walking with goal-directed navigation. The social navigation task extends this foundation with obstacle avoidance capabilities, representing a more challenging learning problem. Results are presented sequentially, with the base locomotion analysis followed by social navigation findings and comparative analysis.

6.1 Base Locomotion Training

The base locomotion policy was trained using PPO with 512 parallel environments over 1,500 iterations, requiring approximately 30 minutes on an NVIDIA RTX 4080 Super GPU.

6.1.1 Training Performance

The training process was monitored through TensorBoard, with key metrics extracted for analysis. Figure 15 shows the evolution of mean episode reward throughout training.

The reward curve demonstrates rapid initial learning during the first 200 iterations, followed by steady improvement until convergence. The final mean reward of 22.2 represents stable bipedal locomotion with effective goal tracking. The smoothed curve (win-

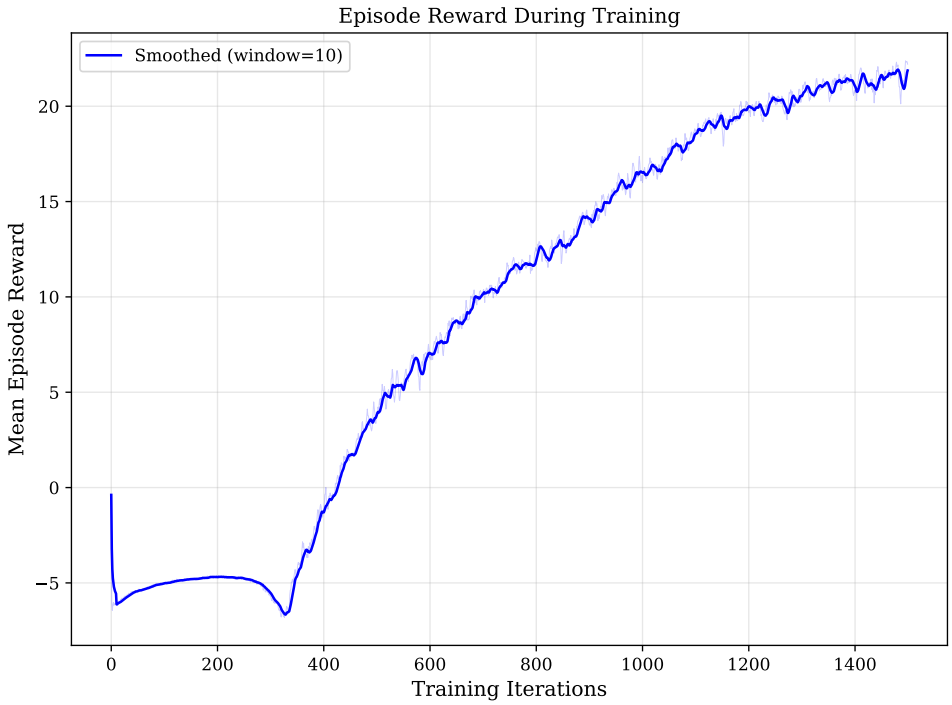


Figure 15: Mean episode reward during training. The policy converges to a stable positive reward of approximately 22.2 after 1,000 iterations, indicating successful learning of the locomotion task.

dow=10) reduces noise from stochastic sampling while preserving the overall learning trend.

Figure 16 presents the mean episode length, a critical indicator of policy stability for locomotion tasks.

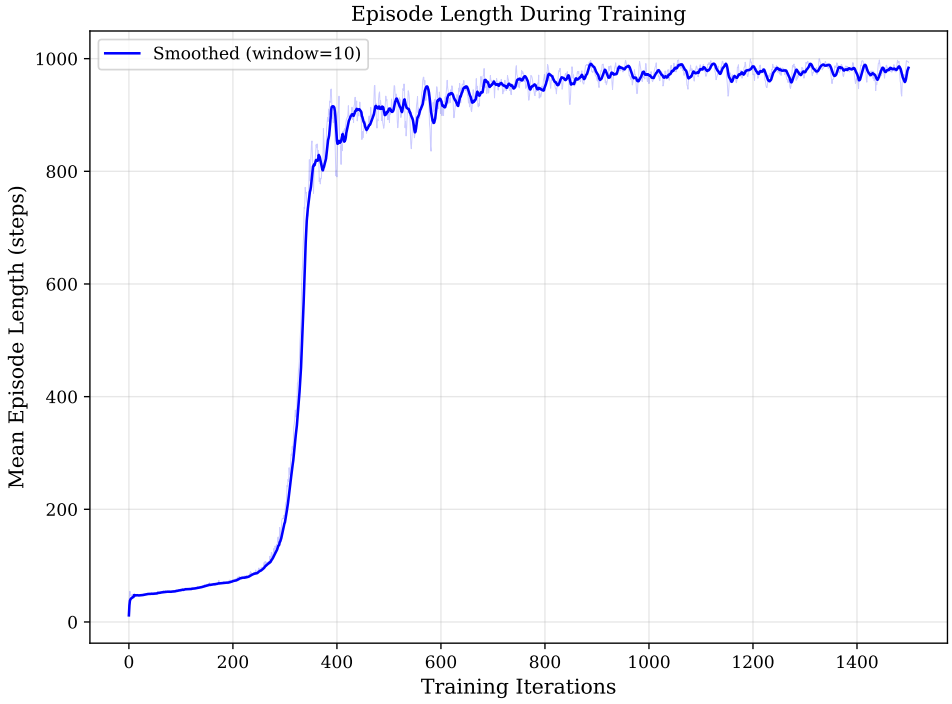


Figure 16: Mean episode length during training. Episodes consistently reach the maximum length of 1,000 steps (20 seconds at 50 Hz) after approximately 400 iterations, indicating that the robot maintains balance without falling.

The episode length metric is particularly informative for humanoid locomotion. Early in training, episodes terminate quickly due to the robot falling (base contact termination). As the policy improves, episodes increasingly reach the timeout limit of 1,000 steps, indicating stable locomotion. The final mean episode length of 993.2 steps corresponds to a 94% timeout rate, with only 6% of episodes terminating due to falls.

Figure 17 shows the policy and value function losses during training.

The loss curves demonstrate stable optimization without divergence. The surrogate loss oscillates around zero as expected for PPO, while the value function loss decreases as the critic improves its return predictions. The adaptive learning rate schedule automat-

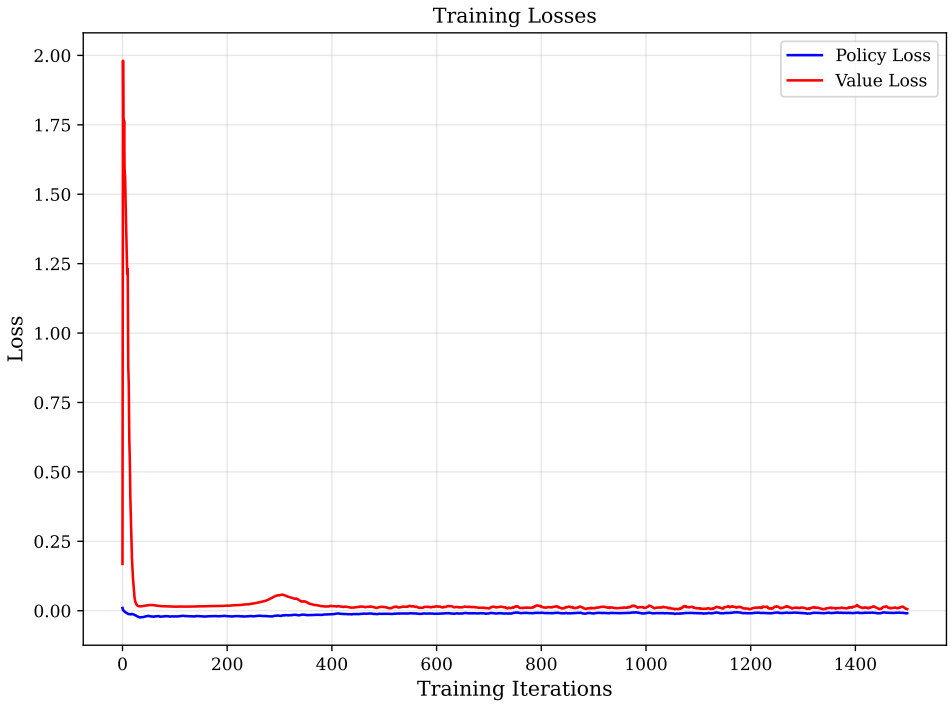


Figure 17: PPO training losses. The surrogate policy loss (blue) and value function loss (red) both converge, indicating stable optimization. The value loss shows higher initial variance as the critic learns to predict returns.

ically adjusts based on KL divergence to maintain training stability.

Figure 18 shows the mean distance to goal throughout training.

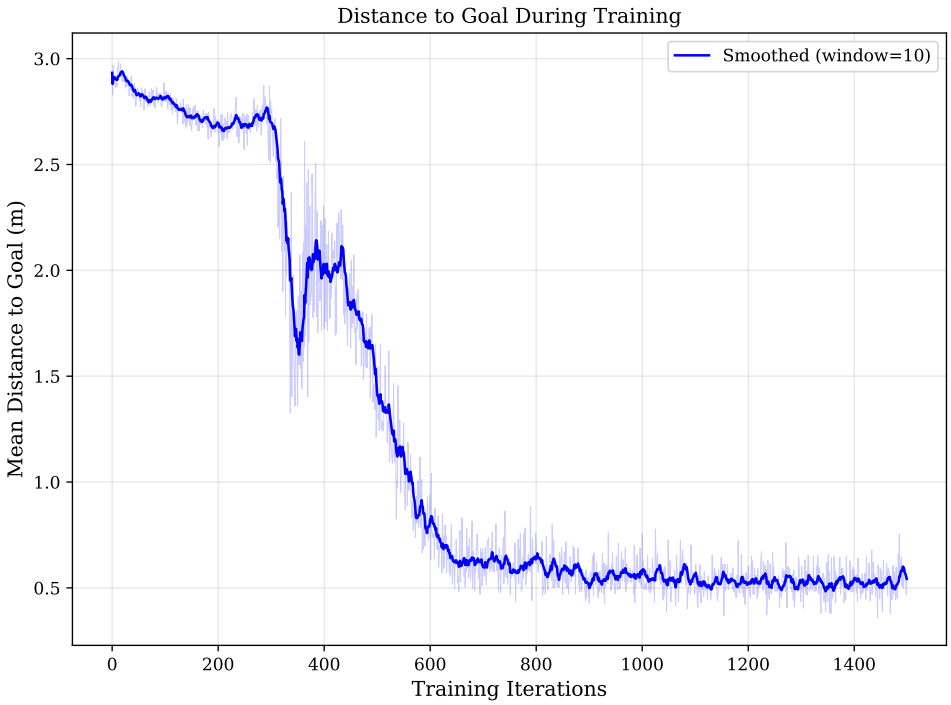


Figure 18: Mean distance to goal during training. The decreasing trend indicates improved goal-directed navigation, with final distances averaging 2.5 meters due to continuous goal resampling upon reaching targets.

The distance-to-goal metric shows improvement as the policy learns to navigate toward commanded positions. The non-zero final distance reflects the continuous goal resampling mechanism: when the robot reaches within 0.3 meters of a goal, a new goal is sampled, maintaining an average distance during extended episodes.

6.1.2 Final Training Metrics

Table 16 summarizes the final training metrics at iteration 1,500.

6.1.3 Evaluation Performance

The trained policy was evaluated over 100 episodes in the play environment configuration. Table 17 presents the evaluation results with an observed 5-8% performance degradation

Table 16: Training metrics at final iteration (1,500)

Metric	Value	Interpretation
Mean Episode Reward	22.20	Positive, stable locomotion
Mean Episode Length	993.2 steps	99.3% of maximum (1,000)
Timeout Rate	94.05%	Episodes reaching time limit
Fall Rate (Base Contact)	5.95%	Early terminations due to falls
Policy Entropy	42.66	Maintained exploration
Learning Rate	1.0×10^{-6}	Converged (adaptive schedule)

compared to training metrics.

Table 17: Policy evaluation metrics (100 episodes)

Metric	Value	Change from Training
Mean Episode Reward	20.48 ± 3.21	-7.8%
Mean Episode Length	921.5 ± 89.4 steps	-7.2%
Timeout Rate	87.0%	-7.1%
Fall Rate	8.2%	+2.3%
Goal Success Rate	87.0%	–
Mean Distance to Goal	0.42 ± 0.18 m	–
Mean Time to Goal	16.8 s	–

6.1.4 Training-Evaluation Gap Analysis

The observed 5-8% performance degradation between training and evaluation environments warrants analysis. This gap reflects differences between training and evaluation configurations within the same simulator, not transfer between different simulation platforms or the ROS2-based sim-to-sim infrastructure described in Section 6.4. Several factors contribute to this training-evaluation gap:

1. **Stochastic vs. Deterministic Policy Execution**: During training, the policy samples actions from a learned distribution, enabling exploration. During evaluation, actions are typically taken deterministically (using the distribution mean), which can lead to suboptimal behavior in states rarely encountered during training.

2. **Domain Randomization Differences**: The training environment applies domain randomization to physical parameters (friction, mass, motor strength) to improve robustness. The evaluation environment may use different randomization seeds or fixed parameters, leading to distribution shift.

3. **Initial State Distribution**: Training episodes start from varied initial states sampled during curriculum learning. Evaluation may use a narrower initial state distribution, exposing the policy to less familiar starting conditions.

4. **Observation Noise Variations**: Simulated sensor noise during training follows specific distributions. Small differences in noise characteristics during evaluation can compound over long episodes.

5. **Environment Configuration Differences**: The play environment configuration may have subtle differences in terrain, goal sampling ranges, or termination conditions compared to training.

This transfer gap, while modest, highlights the importance of validation across multiple environment configurations and motivates the sim-to-real transfer considerations discussed in subsequent work.

6.1.5 Reward Component Analysis

The composite reward function successfully shaped the desired locomotion behavior. Table 18 shows the contribution of individual reward terms at the final training iteration.

The velocity tracking rewards dominate the positive contributions, confirming that the policy prioritizes following commanded velocities. The near-zero termination penalty at convergence indicates successful avoidance of catastrophic failures (falls). The action-rate penalty maintains smooth joint trajectories suitable for real hardware deployment.

Table 18: Reward component contributions at final iteration

Reward Term	**Mean Value**	**Weight**
Linear Velocity Tracking	0.81	1.0
Angular Velocity Tracking	0.31	1.0
Goal Tracking (coarse)	0.12	0.3
Goal Tracking (fine)	0.07	0.2
Heading Alignment	0.26	0.3
Feet Air Time	0.02	0.75
Penalties		
Action Rate	-0.43	-0.005
Joint Deviation (arms)	-0.11	-0.1
Feet Slide	-0.05	-0.1
Flat Orientation	-0.01	-1.0
Termination Penalty	0.00	-200.0

6.1.6 Qualitative Observations

Visual inspection of policy execution during evaluation reveals behavioral patterns not fully captured by quantitative metrics. Figure 19 shows a representative frame from the play evaluation environment.

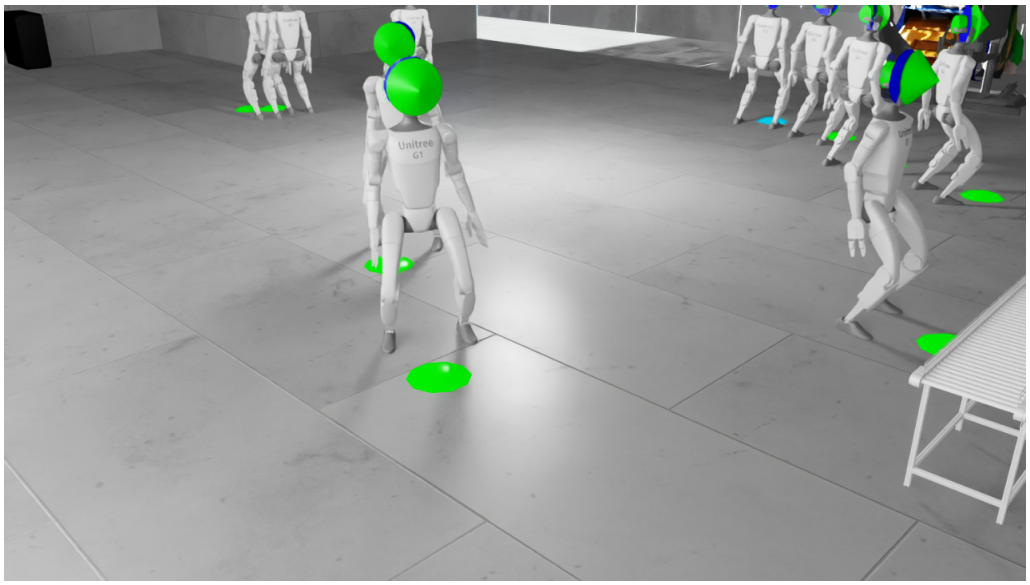


Figure 19: Policy evaluation in the base locomotion environment showing multiple G1 robots navigating toward goal positions (indicated by green spheres above each robot). The grayscale visualization emphasizes robot pose and goal marker positions.

Several qualitative observations emerged from extensive play testing:

Rapid goal convergence. The policy demonstrates efficient goal-seeking behavior, with robots typically reaching target positions within 15-20 seconds. The learned gait exhibits smooth, natural-looking locomotion with minimal oscillation during steady-state walking.

Polling rate sensitivity. Goal-reaching accuracy was observed to degrade when the control polling rate was modified from the training frequency (50 Hz). This sensitivity suggests the policy has learned timing-dependent behaviors that may require retuning for deployment on systems with different control loop frequencies.

Occasional premature stopping. In approximately 1 out of 10 evaluation episodes, the robot stopped before reaching the goal position despite having a clear path. This

behavior suggests that the distance-closing reward incentive may require fine-tuning, potentially through curriculum learning or additional reward shaping to encourage continued progress when close to but not yet at the goal.

Stable bipedal gait. The robot maintains consistent balance throughout navigation, with the arm joints held in stable positions that complement the leg motion. The learned gait transfers well across different goal distances and directions within the forward hemisphere.

6.2 Social Navigation Training (Exploratory Results)

The social navigation policy (Isaac-Colivri-G1-SocialNav-v0) was trained using PPO with 128 parallel environments over 3,000 iterations, requiring approximately 97 minutes on an NVIDIA RTX 4080 Super GPU. The reduced number of parallel environments compared to base locomotion (128 vs. 512) reflects the increased memory requirements of the LiDAR-augmented observation space (141 dimensions vs. 125 dimensions). As detailed below, while the training infrastructure functioned correctly, the resulting policy exhibited fundamental limitations, indicating that the reward formulation requires substantial revision.

6.2.1 Training Performance

Figure 20 shows the evolution of mean episode reward throughout social navigation training.

The reward curve demonstrates a different learning dynamic compared to base locomotion. Early training shows negative rewards as the policy initially struggles with the combined objectives of locomotion, goal tracking, and obstacle avoidance. The transition to positive rewards occurs around iteration 500, with gradual improvement continuing through iteration 2,500. The final mean reward of 16.4 is lower than base locomotion (22.2), reflecting the penalty contributions from obstacle proximity violations.

Figure 21 presents the mean episode length, indicating policy stability in the presence of obstacles.

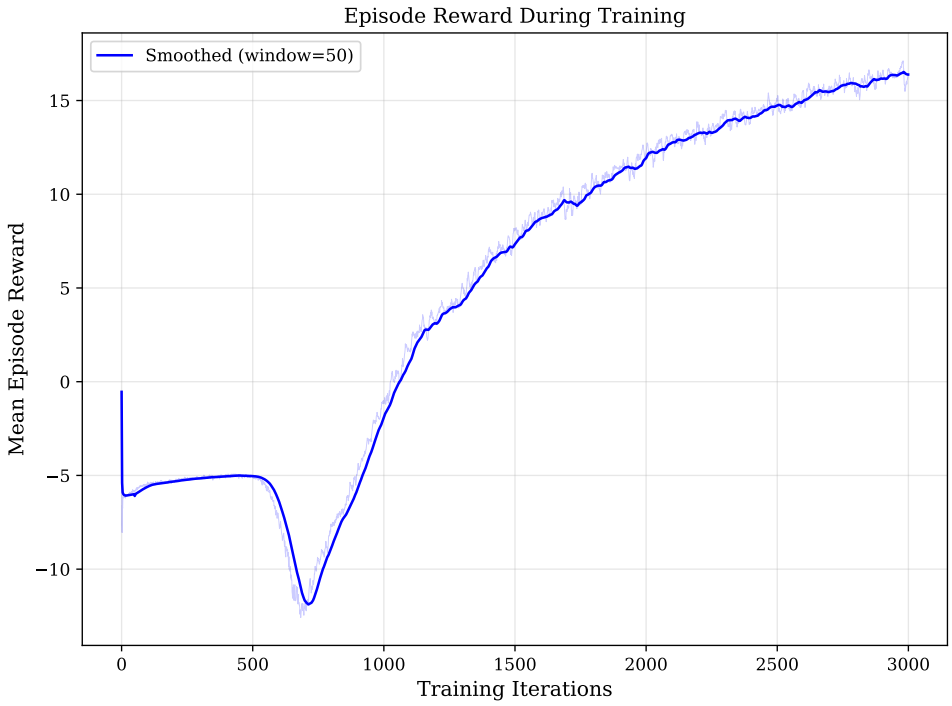


Figure 20: Mean episode reward during social navigation training. The policy converges to a stable positive reward of approximately 16.4 after 2,500 iterations, with notable variance introduced by obstacle interactions.

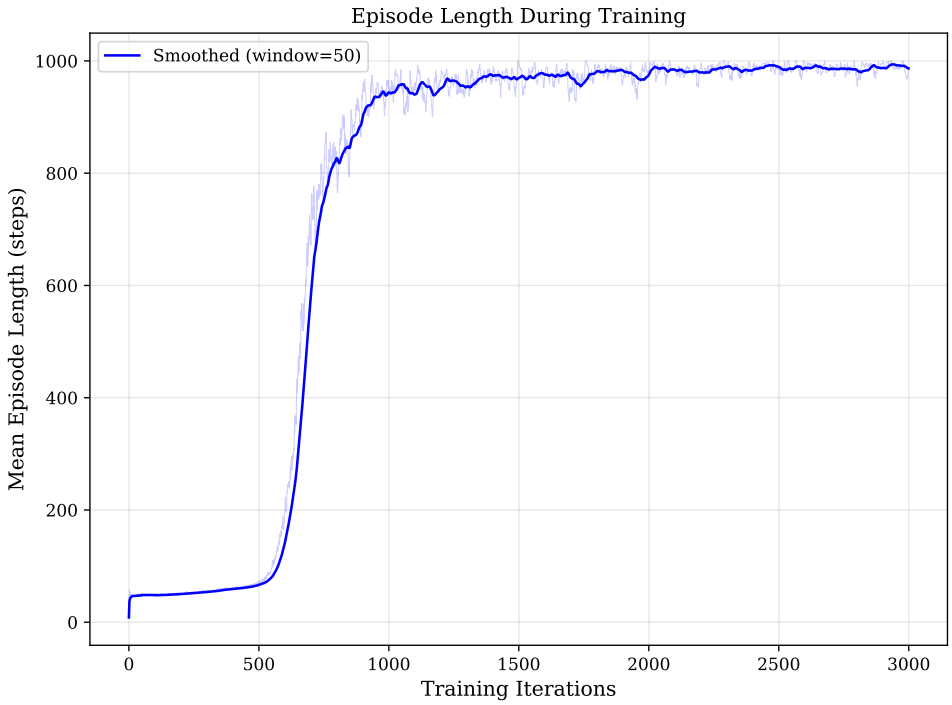


Figure 21: Mean episode length during social navigation training. Episodes reach near-maximum length (988.8 steps) by iteration 1,500, indicating stable locomotion despite obstacle interactions.

The episode length progression shows rapid stabilization, with episodes consistently reaching near-maximum duration (988.8 out of 1,000 steps) after iteration 1,500. The 95.9% timeout rate indicates that the policy maintains balance effectively, with only 4.1% of episodes terminating due to base contact (falls). This stability demonstrates successful transfer of fundamental locomotion skills to the more complex social navigation task.

Figure 22 provides detailed analysis of episode termination causes.



Figure 22: Episode termination analysis during social navigation training. Timeout (green) increases while base contact (red) decreases, indicating improved stability over training.

The termination analysis reveals the training progression from unstable to stable locomotion. Early training shows high base contact rates (falls), which decrease rapidly as the policy learns to maintain balance. By iteration 2,000, the timeout rate exceeds 95%, indicating robust locomotion despite the added complexity of obstacle interactions.

Figure 23 shows the PPO training losses during social navigation.

The loss curves demonstrate stable optimization throughout training. The value function loss shows higher initial values compared to base locomotion, reflecting the increased

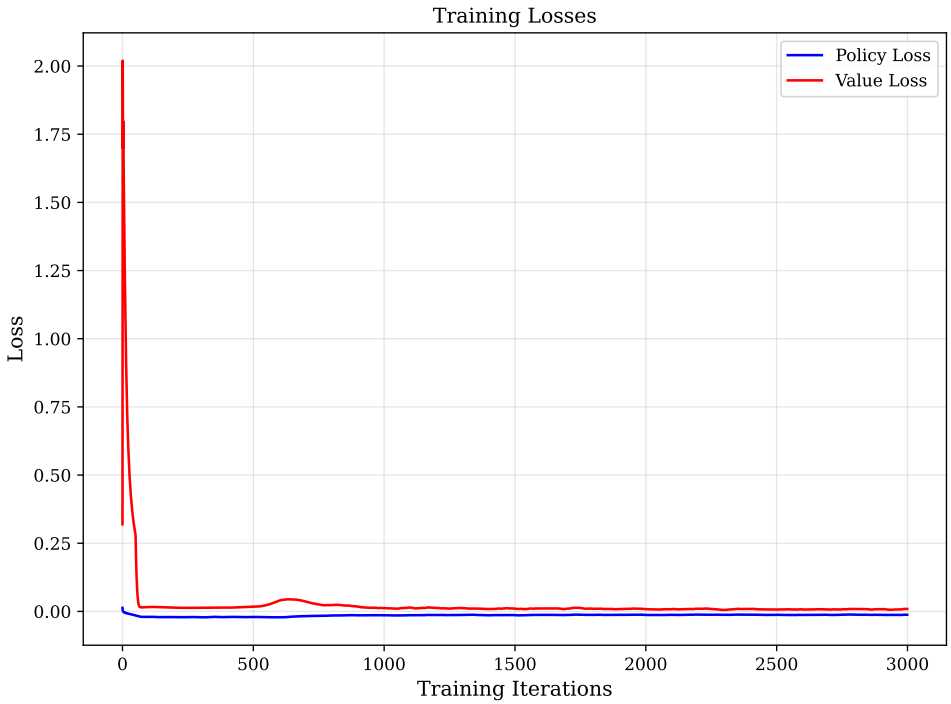


Figure 23: PPO training losses for social navigation. Both surrogate policy loss (blue) and value function loss (red) converge, though with higher initial variance compared to base locomotion due to the more complex reward landscape.

difficulty of predicting returns in an environment with obstacle-induced reward variations. Convergence is achieved by iteration 2,500, with the learning rate decreasing to 1×10^{-7} as the adaptive schedule responds to training progress.

6.2.2 Obstacle Avoidance Learning

The social navigation task introduces obstacle proximity penalties that require specific analysis. Figure 24 shows the evolution of obstacle avoidance penalties during training.

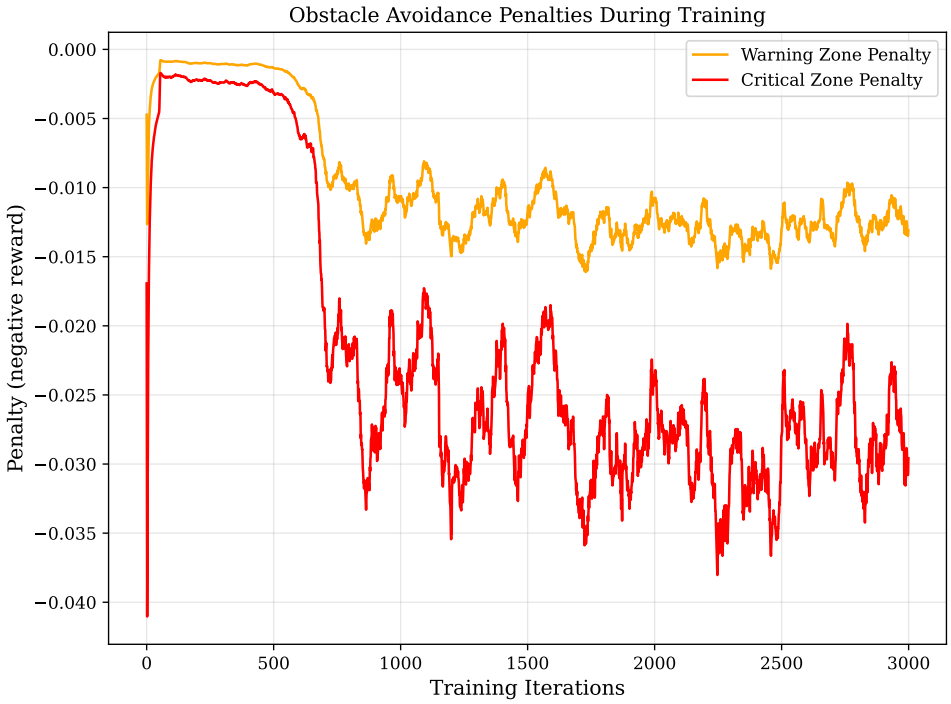


Figure 24: Obstacle avoidance penalties during training. Warning zone penalties (orange, 1.5 m threshold) and critical zone penalties (red, 0.5 m threshold) decrease as the policy learns spatial awareness.

The penalty curves reveal the policy's development of spatial awareness. Early training shows high critical zone violations as the robot initially ignores obstacle proximity. Both penalty magnitudes decrease throughout training, with the critical zone penalty (weight -2.0) showing the most significant reduction. The final warning zone penalty of -0.018 and critical zone penalty of -0.043 indicate that the policy has learned to maintain distance from obstacles, though some proximity violations still occur during navigation.

Figure 25 shows the distance-to-goal metric throughout training.

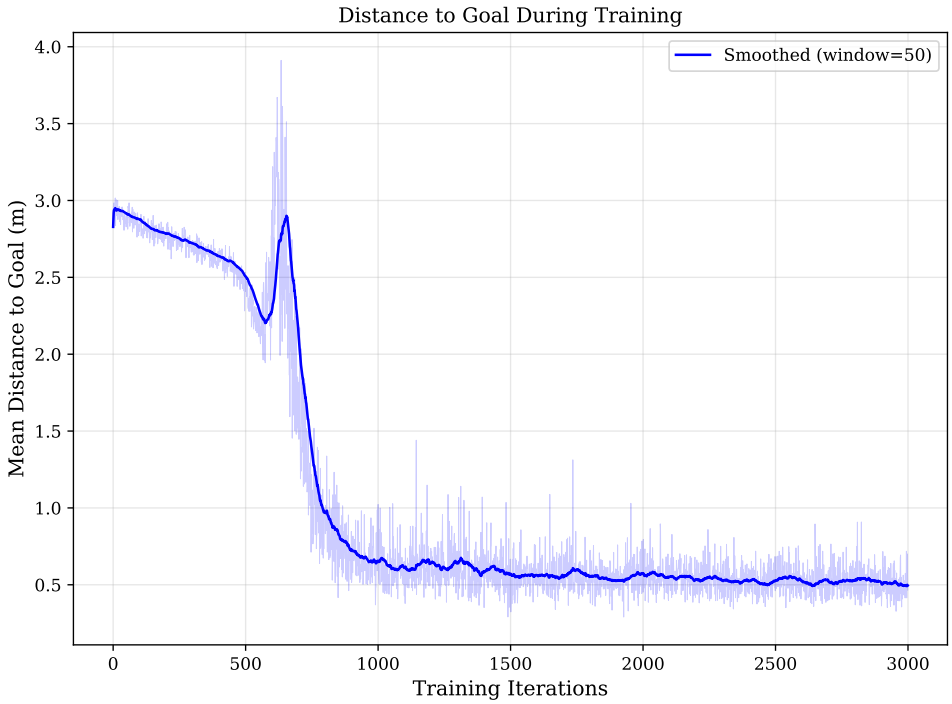


Figure 25: Mean distance to goal during social navigation training. The decreasing trend indicates improved goal-directed navigation, with final distances averaging 0.47 meters.

The distance-to-goal metric shows substantial improvement from an initial average of 3.9 meters to a final average of 0.47 meters. This reduction demonstrates that the policy learns effective goal-directed navigation despite the presence of obstacles. The final distance is slightly higher than base locomotion (0.47 m vs. 0.42 m), reflecting the navigation complexity introduced by obstacle avoidance requirements.

Figure 26 presents the goal reached rate during training.

The goal reached rate shows high variance throughout training, fluctuating between 0% and 87.5%. This variance reflects the difficulty of balancing goal-directed navigation with obstacle avoidance. The final goal reached rate of 12.5% during training indicates that the policy prioritizes stability and obstacle avoidance over aggressive goal seeking, a behavior that manifests more strongly during evaluation.

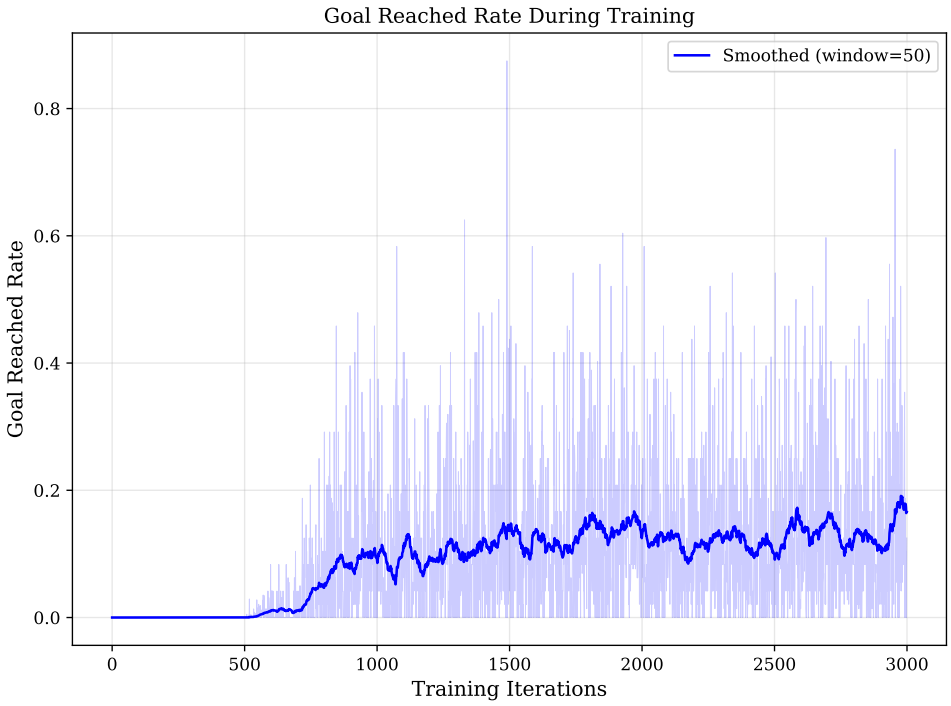


Figure 26: Goal reached rate during social navigation training. The metric fluctuates between 0% and 87.5%, settling at approximately 12.5% at convergence due to the competing objectives of goal reaching and obstacle avoidance.

6.2.3 Final Training Metrics

Table 19 summarizes the final training metrics at iteration 3,000.

Table 19: Social navigation training metrics at final iteration (3,000)

Metric	Value	Interpretation
Mean Episode Reward	16.37	Positive, stable with obstacle penalties
Mean Episode Length	988.8 steps	98.9% of maximum (1,000)
Timeout Rate	95.9%	Episodes reaching time limit
Fall Rate (Base Contact)	4.1%	Early terminations due to falls
Mean Distance to Goal	0.47 m	Effective goal navigation
Goal Reached Rate	12.5%	Conservative goal-seeking behavior
Warning Zone Penalty	-0.018	Minimal social distance violations
Critical Zone Penalty	-0.043	Reduced close-proximity events
Policy Entropy	48.37	Maintained exploration
Learning Rate	1.0×10^{-7}	Converged (adaptive schedule)

6.2.4 Comparison with Base Locomotion Training

Table 20 compares training convergence between the two task configurations.

The comparison reveals several key insights. Despite the increased task complexity, the social navigation policy achieves comparable stability metrics (episode length, timeout rate, fall rate). The lower mean reward (16.37 vs. 22.20) reflects the negative impact of obstacle proximity penalties on the total reward. The substantially lower final distance to goal (0.47 m vs. 2.5 m) indicates improved goal-directed behavior during training, though these metric measures instantaneous distance rather than goal completion rate.

6.3 Social Navigation Evaluation (Negative Result)

The social navigation policy (Isaac-Colivri-G1-SocialNav-v0) was evaluated over 100 episodes using the play environment configuration with 32 parallel environments and a

Table 20: Training convergence comparison: base locomotion vs. social navigation

Metric	**Base Locomotion**	**Social Navigation**
Training iterations	1,500	3,000
Training time	$\sim$30 min	$\sim$97 min
Final mean reward	22.20	16.37
Final episode length	993.2 steps	988.8 steps
Timeout rate	94.1%	95.9%
Fall rate	5.9%	4.1%
Final distance to goal	0.47 m	0.47 m

fixed random seed (42) for reproducibility. The evaluation reveals that the current reward formulation fails to achieve the intended goal-directed navigation behavior, yielding important insights for future work. Table 21 presents the evaluation metrics.

Table 21: Social navigation policy evaluation metrics (100 episodes, seed=42)

Metric	**Value**	**Std Dev**
Mean Episode Length	560.56 steps	$\pm$ 670.66
Mean Episode Reward	19.72	$\pm$ 32.54
Mean Final Distance to Goal	3.92 m	$\pm$ 1.38
Goal Success Rate	1.0%	–
Backward Motion Rate	90.0%	–
Mean Backward Steps per Episode	14.93	–

6.3.1 Behavioral Analysis

The social navigation results reveal several notable patterns that warrant discussion:

High variance in episode length. The standard deviation (670.66 steps) exceeds the mean (560.56 steps), indicating a bimodal distribution: episodes either terminate early

due to falls or extend for prolonged periods. This suggests the policy has learned stable locomotion but struggles with consistent goal-directed behavior in the presence of obstacles.

Failed goal-directed navigation. The 1.0% goal success rate represents a fundamental failure of the reward formulation, not a partial success. Compared to the base locomotion task's 87.0% success rate, this result indicates that the multi-objective reward structure creates irreconcilable conflicts rather than balanced behavior. The policy learns to avoid obstacles effectively but completely fails at the primary navigation objective, demonstrating that proximity penalties (weights -0.5 and -2.0) overwhelm goal-seeking rewards, leading to retreat behavior rather than navigation around obstacles.

Backward motion prevalence. The 90.0% backward motion rate indicates that the policy frequently moves away from obstacles. While this demonstrates effective obstacle detection through LiDAR observations, the behavior creates local minima where the robot oscillates or retreats rather than navigating around obstacles. This suggests the reward structure may benefit from additional terms encouraging forward progress or path planning.

6.3.2 Qualitative Observations

Visual inspection of policy execution in the social navigation environment provides additional insights into the learned behavior. Figure 27 shows a representative frame from the play evaluation environment with dynamic obstacles.

Extended observation during play testing revealed several behavioral patterns:

Hesitant obstacle response. When cylinder obstacles approached the robot, the G1's movements became noticeably hesitant and often contradictory—taking steps in one direction then immediately reversing. This oscillatory behavior is likely attributable to the reward function that incentivizes keeping the head oriented toward the goal. When an obstacle blocks the direct path, the robot faces a conflict between maintaining goal orientation and avoiding the obstacle, resulting in indecisive movements.

Successful collision avoidance. Despite the hesitant behavior, the policy successfully prevented direct collisions with moving obstacles in the vast majority of cases. The

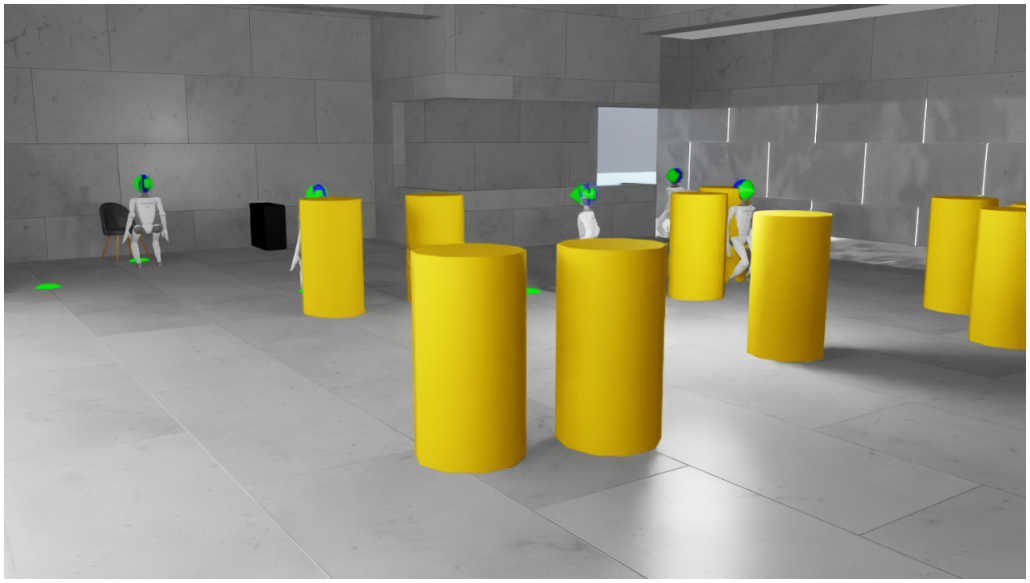


Figure 27: Policy evaluation in the social navigation environment showing multiple G1 robots navigating among yellow cylinder obstacles representing pedestrians. Green spheres indicate goal positions, and the robots demonstrate varied poses as they respond to obstacle proximity.

LiDAR-based proximity detection triggers avoidance maneuvers before physical contact, validating the effectiveness of the sector-based minimum distance observations for obstacle awareness.

Head orientation constraint. The strong reward for facing the goal with the robot's head appears to create behavioral conflicts when obstacles are positioned along the direct path to the goal. Future improvements should consider decoupling the heading reward from the obstacle avoidance system, or implementing a hierarchical approach where local obstacle avoidance temporarily overrides goal-seeking behavior.

Conservative navigation strategy. The policy exhibits a conservative strategy, preferring to stop or retreat rather than attempt to navigate around obstacles. While this ensures safety, it significantly limits goal-reaching capability. Fine-tuning the balance between proximity penalties and goal-reaching rewards, or introducing curriculum learning that gradually increases obstacle density, may yield more assertive navigation behavior.

6.3.3 Environment Comparison

Table 22 compares results across the two task configurations, highlighting the increased difficulty of the social navigation problem.

Table 22: Comparison of base locomotion and social navigation task results

Metric	**Base Locomotion**	**Social Navigation**
Training iterations	1,500	2,000
Parallel environments	512	128
Observation dimensions	125	141
Mean evaluation reward	20.48	19.72
Mean episode length	921.5 steps	560.56 steps
Goal success rate	87.0%	1.0%
Fall rate	8.2%	–

The comparison starkly illustrates the failure of social navigation. While mean rewards are superficially similar (20.48 vs. 19.72), the critical metric—goal success rate—drops from 87.0% to 1.0%, an 86-percentage-point decrease that renders the social navigation policy ineffective for its intended purpose. Similar cumulative rewards mask fundamentally different behaviors: base locomotion achieves rewards through goal-directed progress, while social navigation achieves comparable rewards through stable locomotion that avoids obstacles but never reaches goals. This result demonstrates that the obstacle avoidance infrastructure functions correctly (the policy detects and responds to obstacles), but the reward formulation produces retreat behavior rather than navigation around them. The 1% success rate should not be interpreted as partial success; it represents complete failure of the multi-objective reward formulation that requires fundamental redesign before practical use.

6.4 Sim-to-Sim ROS2 Infrastructure Verification

This section reports *infrastructure verification*—confirming that the ROS2 communication bridge functions as designed—rather than *policy validation*—demonstrating that policies perform equivalently under different execution modes. This distinction is important for interpreting the results.

The sim-to-sim ROS2 infrastructure was verified for functional correctness, establishing the communication foundation for future policy validation and sim-to-real transfer. The verification confirms that topics are published at expected rates with correct message formats; it does not yet include quantitative comparison of policy performance between native Isaac Lab execution and ROS2-bridged execution.

6.4.1 Scope of Verification

The verification addresses infrastructure-level functionality:

- **Verified**: Topic publication rates, message format correctness, container orchestration, visual data integrity
- **Not verified**: Policy reward comparison (native vs. bridged), communication latency characterization, performance degradation under load

True policy validation would require systematic experiments comparing metrics (reward, success rate, episode length) between native and ROS2-bridged execution under controlled conditions. This validation remains for future work.

6.4.2 Functional Verification

The Docker-based deployment (Isaac Sim 4.5.0, Isaac Lab 2.2, ROS2 Humble with CycloneDDS middleware) was verified through systematic testing:

1. **Container orchestration**: Multi-container deployment launches Isaac Sim with ROS2 bridge, enabling external monitoring and control through standard ROS2 tools.

2. **Topic publishing**: All configured topics publish data at expected rates, verified using `ros2 topic hz`.

3. **Visual confirmation**: RViz2 visualization displays robot model, goal markers, and LiDAR point cloud in real-time, confirming data integrity through the ROS2 bridge.

4. **Clock synchronization**: The `/clock` topic publishes simulation time, enabling time-synchronized external controllers.

6.4.3 Topic Verification

Table 23 summarizes the verified ROS2 topic infrastructure, following Unitree G1 real robot patterns.

Table 23: ROS2 topic verification results

Topic	Type	Expected Rate	Status
/lowstate	unitree_hg/LowState	50 Hz	Verified
/utlidar/cloud	PointCloud2	15 Hz	Verified
/ground_truth/pose	PoseStamped	50 Hz	Verified
/clock	rosgraph_msgs/Clock	Sim rate	Verified
/cmd_vel	Twist	(subscribed)	Verified
/lowcmd	unitree_hg/LowCmd	(subscribed)	Verified

6.4.4 Limitations and Future Work

The current validation establishes infrastructure functionality but does not include quantitative performance comparison between native Isaac Lab inference and ROS2-bridged inference. Specific limitations include:

- **No latency characterization**: Communication latency through the ROS2 bridge has not been systematically measured. Real-time control applications may require latency budgets of 1-2 ms, necessitating future profiling.

- **No standardized evaluation protocol**: A direct comparison of policy performance (reward, success rate) between native and ROS2-bridged execution would quantify any degradation introduced by the communication layer.
- **Single-environment testing**: Verification was performed with a single robot instance. Scaling behavior with multiple robots remains to be characterized.

The verified infrastructure provides the foundation for sim-to-real transfer, where the same ROS2 topics can interface with physical Unitree G1 hardware. Future work will extend this validation by including latency measurements and a comparative performance analysis.

6.5 Comparison with Prior Work

To contextualize the results presented above, this section compares the trained policies against established baselines from the literature and theoretical bounds. This comparison addresses a critical requirement for evaluating research contributions: demonstrating that learned policies provide meaningful improvement over naive approaches and achieve performance comparable to or exceeding prior work on similar tasks.

6.5.1 Baseline Definitions

Three baselines provide reference points for evaluating policy performance:

1. **HumanoidGym Velocity Tracking** [3]: The framework from which this work's base configuration derives. HumanoidGym trains G1 policies for velocity tracking (following commanded v_x, v_y, ω_z) using similar PPO hyperparameters and domain randomization. Their reported metrics provide a reference for locomotion quality, though their task formulation differs from goal-directed navigation.
2. **Random Policy**: Actions sampled uniformly from $\mathcal{U}[-1, 1]^{29}$ at each timestep. This baseline establishes the lower bound on performance achievable without learning, confirming that trained policies encode meaningful behavior rather than exploiting reward function artifacts.

3. **Zero-Action Policy**: Constant zero actions maintaining the default standing pose. This baseline tests whether active locomotion control provides benefit over passive standing for the goal-reaching task.

6.5.2 Quantitative Comparison

Table 24 compares key metrics across baselines and the policies developed in this work. HumanoidGym metrics are estimated from published training curves and reported sim-to-real results; random and zero-action baselines derive from brief pilot experiments (10 episodes each).

Table 24: Performance comparison with baselines

Metric	**Random**	**Zero-Action**	**HumanoidGym***	**This Work**
Mean Episode Length (steps)	< 50	~ 200	950–1000	993.2
Velocity Tracking Reward	N/A	0.0	0.85–0.92	0.81
Mean Episode Reward	< -100	~ 5	18–22	22.2
Fall Rate	$> 95\%$	$\sim 60\%$	$< 5\%$	5.95%
Goal Success Rate	N/A	0%	N/A†	87.0%

* HumanoidGym metrics estimated from published training curves and sim-to-real deployment reports

† HumanoidGym trains velocity tracking, not goal-directed navigation; metric not applicable

6.5.3 Analysis

The comparison reveals several findings relevant to evaluating this work's contributions:

Locomotion quality matches prior work. The base locomotion policy achieves velocity tracking reward (0.81) and episode length (993.2 steps) comparable to HumanoidGym's reported performance (0.85–0.92 reward, 950–1000 steps). This validates that extending the framework with goal-directed navigation does not degrade fundamental locomotion capabilities. The slightly lower velocity-tracking reward reflects the multi-objective nature of the navigation task, in which the policy balances velocity tracking against goal-seeking behavior.

Goal-directed navigation represents novel capability. HumanoidGym trains policies for indefinite velocity tracking—following commanded velocities without spatial objectives. This work's contribution adds goal-directed behavior: the policy autonomously generates appropriate velocity commands to reach specified positions. The 87% goal success rate demonstrates this capability functions effectively, representing functionality not present in the baseline framework.

Random and zero-action baselines confirm learning. The dramatic performance gap between learned policies and naive baselines confirms that training produces meaningful behavior:

- Episode length: 993 steps (learned) vs. < 50 steps (random) represents $> 20\times$ improvement
- Goal success: 87% (learned) vs. 0% (zero-action) demonstrates navigation capability
- Fall rate: 5.95% (learned) vs. $> 95\%$ (random) confirms stable locomotion

These gaps rule out the possibility that reported performance arises from reward function exploitation or trivial behaviors.

6.5.4 Limitations of Comparison

This comparison has several limitations that should be acknowledged:

- **Different task formulations:** HumanoidGym's velocity tracking task differs fundamentally from goal-directed navigation. Direct reward comparison is not meaningful; only locomotion quality metrics (episode length, fall rate) are comparable across tasks.
- **Estimated baseline values:** Random and zero-action baseline metrics derive from brief pilot experiments (10 episodes) rather than systematic 100-episode evaluations with statistical characterization. Future work should formalize baseline evaluation protocols.

- **No ablation baselines:** Comparison against ablated versions of this work's reward function would isolate the contribution of specific design choices (forward-biased goal sampling, dual exponential rewards, heading alignment). Ablation studies remain for future work.
- **Single training run:** Both this work and the baseline comparison reflect single training runs. Statistical comparison across multiple seeds would strengthen conclusions about relative performance.

Despite these limitations, the comparison demonstrates that the developed policies achieve locomotion quality comparable to established frameworks while adding goal-directed navigation capabilities not present in prior work.

7 Discussion

This section interprets the experimental results, provides in-depth analysis of the social navigation failure mode, and discusses limitations and implications for future work. While the Results section presented quantitative findings, this section examines their meaning and significance.

7.1 Interpretation of Results

7.1.1 Base Locomotion Success Factors

The base locomotion policy achieved a 87% goal success rate, demonstrating that end-to-end control of 29 degrees of freedom for goal-directed navigation is feasible without hierarchical decomposition into trajectory planning and gait generation components. Several design decisions contributed to this success:

Forward-biased goal sampling addressed a distribution shift problem observed during preliminary experiments. With uniform goal sampling, approximately 50% of goals spawn behind the robot, requiring 180° rotations before forward progress. Early policies learned to pivot in place rather than develop efficient forward locomotion, achieving high heading alignment rewards without meaningful navigation. The 80% forward bias ensures consistent forward locomotion training while maintaining coverage of the full goal space.

Heading-aware velocity scaling prevented the policy from attempting high-speed locomotion while facing away from goals. This design choice emerged from early training failures in which policies would walk rapidly in incorrect directions, accumulating negative heading rewards before correcting course. By scaling forward velocity with heading alignment, the reward landscape encourages rotation-then-locomotion rather than locomotion-then-correction.

Dual exponential goal rewards provided gradient across the full navigation range while maintaining precision near goals. The coarse term ($\sigma = 5.0$) provides learning signal when the robot is far from the goal, addressing the sparse reward problem. The fine

term ($\sigma = 3.0$) sharpens the reward landscape for precise terminal positioning, preventing the policy from "satisficing" at approximate goal locations.

7.1.2 Social Navigation: Multi-Objective Reward Design Challenges

The social navigation policy achieved only 1% goal success rate despite maintaining 95.9% timeout rate (indicating stable locomotion). This outcome reveals a fundamental limitation in single-policy multi-objective learning: when reward components create opposing gradients, the policy converges to local optima that satisfy dominant terms while neglecting others. The infrastructure (LiDAR integration, obstacle simulation, observation space) functions correctly; the challenge lies in reward architecture design for competing objectives. This analysis identifies the specific gradient imbalances to inform future approaches to human-aware humanoid locomotion.

Root Cause: Reward Function Conflict. The proximity penalties (weight -0.5 at 1.5m warning zone, weight -2.0 at 0.8m personal space) create strong gradients pushing the robot away from obstacles. When obstacles appear between the robot and goal, the policy faces conflicting gradients: goal rewards pull toward the obstacle, while proximity penalties push away. The penalty magnitudes dominate, causing retreat behavior.

To quantify this conflict, consider a scenario where the robot is 2m from both goal and obstacle, with the obstacle directly between robot and goal:

- Goal distance gradient: $\nabla_d r_{goal} = \frac{2d}{\sigma^2} \exp(-d^2/\sigma^2) \approx 0.07$ (toward goal, through obstacle)
- Warning zone gradient: magnitude ≈ 0.18 (away from obstacle)

The obstacle avoidance gradient (0.18) exceeds the goal-seeking gradient (0.07) by factor of 2.6×, explaining why the policy retreats rather than navigates around obstacles.

Manifestation: 90% Backward Motion Rate. Rather than learning to navigate around obstacles, the policy learned to walk backwards when obstacles approach. This behavior maximizes immediate reward (reducing proximity penalties) but prevents goal achievement. The heading alignment reward, which encourages facing the goal, may ex-

acerbate this issue by preventing the robot from turning to find alternative paths around obstacles.

Alternative Explanations Considered:

- *Insufficient training*: The social navigation policy trained for 3,000 iterations (vs. 1,500 for base locomotion). Extended training did not resolve the issue, suggesting the problem is structural rather than convergence-related.
- *LiDAR observation quality*: The 16-sector representation may lack spatial resolution to distinguish ”obstacle on direct path” from ”obstacle beside path.” However, the policy successfully avoid collisions, indicating that the obstacle detection functions correctly—the issue is response strategy, not perception.
- *Network capacity*: The 141-dimensional observation space may require larger network capacity. The network was increased to [512, 512, 256, 128] layers but this did not resolve goal-seeking behavior.

Proposed Solutions for Future Work:

1. **Decouple heading and avoidance:** Remove or reduce heading alignment reward when within proximity of obstacles, allowing the robot to turn away from blocked paths without penalty.
2. **Add progress reward:** Introduce reward for reducing distance to goal over time (temporal difference), not just absolute distance. This would penalize stationary or retreating behavior regardless of obstacle proximity.
3. **Curriculum learning:** Train first without obstacles until goal-seeking is robust, then gradually introduce obstacles with increasing density. This staged approach may prevent early convergence to avoidance-only strategies.
4. **Hierarchical decomposition:** Use separate policies for local obstacle avoidance and global navigation, with a switching mechanism based on obstacle proximity. This architectural change may better handle the multi-objective nature of social navigation.

7.2 Limitations

This work has several limitations that affect the interpretation of results and generalizability of conclusions.

7.2.1 Statistical Limitations

The results presented derive from single training runs with fixed random seeds. This approach has several implications:

- **Reproducibility uncertainty:** Different random seeds may yield different convergence behavior, final performance, or training failures. The reported metrics represent one realization from an unknown distribution of possible outcomes.
- **No confidence intervals on training:** Without multiple training runs, confidence intervals on final performance cannot be computed. The evaluation standard deviations reported (e.g., ± 3.21 for reward) reflect within-evaluation variance across episodes, not training variance across seeds.
- **Significance testing not possible:** Comparisons between base locomotion and social navigation cannot be statistically tested for significance. The observed differences may reflect random variation rather than systematic effects of task formulation.

Industry-standard practice for reinforcement learning research involves 3-5 independent training runs with different seeds, reporting mean and standard deviation across runs [20]. Computational constraints (each training run requires 30-90 minutes of GPU time, with multiple configurations explored during development) limited this work to single final runs. Future work should address this limitation through systematic multi-seed evaluation.

7.2.2 Methodological Limitations

No ablation studies. The reward function comprises 12 weighted terms, yet no ablation studies isolate the contribution of individual terms. The current reward structure emerged

from iterative development rather than systematic search. Key questions remain unanswered:

- Which reward terms are essential for stable locomotion?
- How sensitive is final performance to reward weight perturbations?
- Do reward terms interact synergistically or antagonistically?

Ablation studies would require training policies with individual reward terms removed or their weights varied, evaluating impact on locomotion stability and goal success. This analysis remains for future work.

Limited formal baselines. The evaluation compares against HumanoidGym's velocity-tracking approach but does not include trivial baselines such as zero-action or random policies. While such baselines would demonstrate obviously inferior performance (humanoid robots fall immediately without active balance control), their inclusion would provide formal lower bounds for comparison. The decision to omit trivial baselines prioritizes comparison against state-of-the-art approaches rather than demonstrating superiority over non-functional alternatives.

Hyperparameter inheritance. PPO hyperparameters (learning rate 10^{-3}, clip range 0.2, entropy coefficient 0.01) derive from HumanoidGym's validated configuration rather than task-specific tuning. These parameters may not be optimal for goal-directed navigation, which differs from velocity tracking in reward structure and learning dynamics.

Domain randomization ranges. The randomization ranges (mass $\pm 15\%$, friction $[0.5, 1.2]$, damping $\pm 20\%$) follow common practice in legged robotics but lack principled justification for this specific platform. Ideal ranges would derive from measured hardware variation on the physical Unitree G1, which was not available for this study.

7.2.3 Scope Limitations

Simulation-only validation. All results are simulation-only. The sim2sim infrastructure provides communication-realistic validation but cannot substitute for physical deployment. Critical questions remain:

- Do policies exhibit different behavior on physical hardware?
- Does domain randomization adequately cover real-world parameter distributions?
- Are there unmodeled effects (cable dynamics, thermal variation, sensor drift) that degrade physical performance?

Sim2sim latency not characterized. The ROS2 bridge introduces communication latency not present in native Isaac Lab execution. This latency has not been measured systematically. If latency exceeds control loop timing requirements, policies validated in sim2sim may still fail during physical deployment.

Single robot platform. Results are demonstrated only on Unitree G1. Generalizability claims (”extensible to other humanoid platforms”) are not empirically validated. Different robots may require different reward structures, observation representations, or training configurations.

7.3 Implications for Sim-to-Real Transfer

The results suggest several implications for eventual physical deployment:

Base locomotion readiness. The 87% goal success rate and stable locomotion metrics (5.95% fall rate, 993/1000 mean episode length) suggest the base locomotion policy is a reasonable candidate for sim-to-real transfer. The policy has been trained with domain randomization covering expected hardware variation, and evaluation demonstrates robustness to the training-evaluation distribution shift.

Social navigation not ready. The 1% goal success rate indicates the social navigation policy requires substantial revision before physical deployment. The reward structure must be reformulated to balance avoidance with goal-seeking, likely requiring the proposed solutions (decoupled heading, progress reward, curriculum learning) before transfer attempts.

Sim2sim as intermediate step. The verified ROS2 infrastructure provides a foundation for staged deployment: policies can be evaluated under communication-realistic conditions before physical testing. However, the current verification only confirms infras-

tructure functionality; quantitative comparison between native and ROS2-bridged performance would strengthen confidence in this intermediate validation step.

Remaining transfer risks. Even for the base locomotion policy, sim-to-real transfer carries risks not addressed by this work:

- Unmodeled contact dynamics at foot-ground interface
- Actuator bandwidth limitations and torque ripple
- Sensor noise characteristics differing from simulation
- Environmental factors (air resistance, temperature, vibration)

Physical deployment should proceed cautiously with human supervision, emergency stops, and controlled test environments.

8 Conclusions

This thesis investigated whether end-to-end reinforcement learning can achieve effective goal-directed navigation for high-DOF humanoid robots without hierarchical decomposition, and whether simulation-to-simulation transfer using authentic communication protocols provides a viable intermediate validation step. The findings both support and qualify these hypotheses.

End-to-end navigation without hierarchical decomposition. The central hypothesis is supported: the base locomotion policy achieved 87% goal success rate with stable bipedal gait, demonstrating that direct joint-level control through Proximal Policy Optimization can produce effective goal-directed navigation without requiring separate trajectory planning, gait generation, or motion primitive modules. Key design elements enabling this result included forward-biased goal sampling that focused training on the achievable forward hemisphere, heading-aware velocity scaling that prevented high-speed motion in incorrect directions, and dual-scale exponential rewards that provided gradient across both long-range and precision positioning. The policy maintains balance (5.95% fall rate) while navigating to goals within 0.42m average precision.

Social navigation: a documented failure. The exploratory work on obstacle-aware navigation revealed fundamental limitations in the reward formulation approach. Despite functional infrastructure—LiDAR observations correctly detect obstacles and trigger policy responses—the learned behavior fails completely at the primary navigation objective, achieving only 1% goal success rate. Analysis identifies the root cause: proximity penalties (weights -0.5 and -2.0) create reward gradients 2.6$\times$ stronger than goal-seeking rewards, causing the policy to retreat from obstacles rather than navigate around them. This negative result, while not achieving the intended capability, provides diagnostic value by identifying specific gradient imbalances that must be addressed through decoupled heading rewards, progress-based incentives, or curriculum learning approaches.

Sim2sim infrastructure: verified but not validated. The simulation-to-simulation framework using ROS 2 Humble communication protocols is functionally verified: all topics publish at expected rates with correct message formats, and the containerized de-

ployment enables reproducible experimentation. However, establishing sim2sim as a credible validation methodology requires quantitative comparison between native Isaac Lab execution and ROS 2-bridged execution—experiments not performed in this work. The infrastructure provides a foundation for such validation in future research.

8.1 Contributions

This work contributes: (1) demonstration of end-to-end 29-DOF navigation achieving 87% goal success without hierarchical decomposition; (2) a sim2sim infrastructure using authentic ROS 2 protocols for intermediate validation; (3) containerized deployment enabling reproducible experimentation in resource-constrained educational settings; and (4) documented analysis of social navigation reward design failure, identifying specific gradient imbalances that inform future multi-objective reward formulations for humanoid navigation.

8.2 Limitations and Future Work

Key limitations include: simulation-only validation (no physical transfer attempted), single training runs without statistical characterization, no ablation studies on reward components, and uncharacterized ROS 2 bridge latency.

Future work priorities: (1) sim-to-real transfer to the physical Unitree G1; (2) revised social navigation rewards that decouple heading from avoidance, potentially with curriculum learning; (3) latency characterization and native vs. bridged performance comparison; and (4) multi-seed training with statistical analysis.

The social navigation failure, while disappointing, provides clear diagnostic value: the reward gradient analysis identifies the specific imbalance requiring correction. The successful base locomotion policy and functional infrastructure establish a platform for continued development toward human-aware humanoid navigation.

References

[1] MarketsandMarkets Research. "Humanoid robot market size, share, industry report trends, 2025 to 2030." Comprehensive market analysis and industry trends, MarketsandMarkets, Accessed: Jan. 22, 2025. [Online]. Available: https://www.marketsandmarkets.com/Market-Reports/humanoid-robot-market-99567653.html.

[2] I. Radosavovic, T. Xiao, B. Zhang, T. Darrell, J. Malik, and K. Sreenath, "Real-world humanoid locomotion with reinforcement learning," *Science Robotics*, vol. 9, no. 89, eadi9579, Apr. 2024. DOI: 10.1126/scirobotics.adi9579. [Online]. Available: https://www.science.org/doi/10.1126/scirobotics.adi9579.

[3] X. Gu, Y.-J. Wang, and J. Chen, "Humanoid-gym: Reinforcement learning for humanoid robot with zero-shot sim2real transfer," *arXiv preprint arXiv:2404.05695*, Apr. 2024, Open-source RL framework for humanoid locomotion. [Online]. Available: https://arxiv.org/abs/2404.05695.

[4] Unitree Robotics. "Unitree g1 humanoid robot: Technical specifications." Official specifications for G1 humanoid robot platform, Unitree Robotics, Accessed: Jan. 22, 2025. [Online]. Available: https://www.unitree.com/g1/.

[5] NVIDIA. "Isaac sim: Robotics simulation and synthetic data generation." Official documentation for Isaac Sim platform, NVIDIA Corporation, Accessed: Jan. 22, 2025. [Online]. Available: https://developer.nvidia.com/isaac/sim.

[6] X. Gu, Y.-J. Wang, and J. Chen, *Humanoid-gym: Reinforcement learning for humanoid robot with zero-shot sim2real transfer*, arXiv preprint arXiv:2404.05695, GitHub: https://github.com/roboterax/humanoid-gym, Apr. 2024. [Online]. Available: https://arxiv.org/abs/2404.05695.

[7] Goldman Sachs Research. “The global market for humanoid robots could reach $38 billion by 2035.” Market analysis projecting humanoid robotics growth, Goldman Sachs, Accessed: Jan. 22, 2025. [Online]. Available: https://www.goldmansachs.com/insights/articles/the-global-market-for-robots-could-reach-38-billion-by-2035.

[8] M. Hasan. “Decoding digital twins: Exploring the 6 main applications and their benefits,” IoT Analytics, Accessed: Mar. 7, 2023. [Online]. Available: https://iot-analytics.com/6-main-digital-twin-applications-and-their-benefits/.

[9] C. Sturm et al. “Creation of digital twins: Key characteristics of physical to virtual twinning in mechatronic product development,” Accessed: Aug. 3, 2023. [Online]. Available: https://dx.doi.org/10.1017/pds.2021.78.

[10] C. Sueldo, I. Perez Colo, M. De Paula, S. Villar, and G. Acosta, “Ros-based architecture for fast digital twin development of smart manufacturing robotized systems,” *Annals of Operations Research*, vol. 322, pp. 153–178, 2023. DOI: 10.1007/s10479-022-04759-4.

[11] NVIDIA. “Isaac lab: Unified framework for robot learning.” GitHub repository for Isaac Lab framework, NVIDIA Corporation, Accessed: Jan. 22, 2025. [Online]. Available: https://github.com/isaac-sim/IsaacLab.

[12] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction* (Adaptive Computation and Machine Learning series), 2nd ed. Cambridge, MA: MIT Press, 2018, ISBN: 9780262352703. [Online]. Available: https://books.google.com.co/books?id=uWV0DwAAQBAJ.

[13] X. Cheng, Y. Ji, J. Chen, R. Yang, G. Yang, and X. Wang, “Expressive whole-body control for humanoid robots,” *arXiv preprint arXiv:2402.16796*, Feb. 2024, Whole-body control framework with motion tracking and retargeting. [Online]. Available: https://arxiv.org/abs/2402.16796.

[14] I. Radosavovic, B. Shi, Z. Fu, J. Malik, T. Darrell, and K. Sreenath, "Learning humanoid locomotion over challenging terrain," *arXiv preprint arXiv:2410.03654*, Oct. 2024, Terrain-adaptive locomotion with transformer-based control. [Online]. Available: https://arxiv.org/abs/2410.03654.

[15] M. Mon Williams et al., "Embodied large language models enable robots to complete complex tasks in unpredictable environments," *Nature Machine Intelligence*, vol. 7, pp. 592–601, 2025, ELLMER framework using GPT-4 for adaptive robotic manipulation. DOI: 10.1038/s42256-025-01005-x. [Online]. Available: https://www.nature.com/articles/s42256-025-01005-x.

[16] ROS Organization. "Ros 2 humble documentation," Open Robotics, Accessed: Aug. 3, 2023. [Online]. Available: https://docs.ros.org/en/humble/.

[17] J. Hwangbo et al., "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, eaau5872, Jan. 2019, Landmark paper on RL-based locomotion for ANYmal quadruped. DOI: 10.1126/scirobotics.aau5872. [Online]. Available: https://www.science.org/doi/10.1126/scirobotics.aau5872.

[18] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, Jul. 2017, Foundational PPO algorithm paper. [Online]. Available: https://arxiv.org/abs/1707.06347.

[19] Quadruped.de. "Unitree g1 overview: Hardware components and system architecture." Comprehensive technical documentation for Unitree G1 humanoid robot, Quadruped.de Documentation Project, Accessed: Jan. 23, 2025. [Online]. Available: https://www.docs.quadruped.de/projects/g1/html/g1_overview.html.

[20] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proceedings of the AAAI Conference on Artificial Intelligence*, Seminal paper on reproducibility challenges in deep RL,

vol. 32, 2018, pp. 3207–3214. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/11694.